



German International University
Faculty of Informatics and Computer Science

VisionBlocks: A Block-Based Learning Platform for Computer Vision Pipelines

Bachelor Thesis

Author: Abdulrahman Mohammed Alawbathani
Supervisors: Dr. Nada Sharaf
Eng. Reham Ayman
Submission Date: 5 January, 2026

This is to certify that:

- (i) the thesis comprises only my original work toward the Bachelor Degree
- (ii) due acknowledgement has been made in the text to all other material used
- (iii) Artificial intelligence tools were used exclusively for language refinement and presentation purposes. No AI tools were used to generate research ideas, methodology, results, analysis, or conclusions. The author remains fully responsible for all content of this thesis.

Abdulrahman Mohammed Alawbathani
5 January, 2026

Acknowledgments

First and foremost, I am deeply grateful to Allah, whose guidance, blessings, and support gave me the strength, patience, and perseverance to complete this thesis.

I want to thank my supervisors, Dr. Nada Sharaf and Eng. Reham Ayman, for their continuous help and guidance with this thesis. Their contributions were invaluable to the completion of the thesis.

I would also like to thank the German International University (GIU) for helping to create the academic environment that has made this research possible.

My heartfelt thanks go to my family and friends for their unwavering support. In particular, I am profoundly thankful to my mother and father, whose constant encouragement, understanding, and emotional support made it possible for me to continue and complete this journey.

Finally, I would like to thank my friend Mohamed Gamal for his support throughout the semester. His encouragement and support helped me navigate the challenges of this period.

Abstract

Computer Vision is a significant area of AI and is widely used nowadays. However, learning computer vision as a beginner is challenging, as there are various steps involved in learning computer vision, like data handling, image preprocessing, and creating and training a model. Image preprocessing is a stage in computer vision that is often treated as a technical step that students follow mechanically, without fully comprehending the significance of image preprocessing. This thesis presents VisionBlocks, a block-based learning environment. This environment is aimed at university-level first-year students who have little to no experience with computer vision. VisionBlocks is a learning environment designed to help novices gain a better understanding of the computer vision pipeline, with a focus on image preprocessing. The system is designed to make learners think about the processes performed on images before training a model. VisionBlocks was tested using a between-groups experimental design and first-year university students. An experimental group learned preprocessing concepts using VisionBlocks, and a control group learned using a video. Learning gain, engagement, and usability were all measured. The results show that there is no statistically significant difference in the learning gain between the two groups in terms of learning preprocessing concepts. Students who used VisionBlocks reported higher levels of engagement with a P-value of < 0.05 , and the result of the usability test shows that the system is perceived as being easy to use. This suggests that focusing on image preprocessing concepts in an interactive way has potential in facilitating effective and engaging learning.

Contents

1	Introduction	1
2	Literature review	3
2.1	Mapping the Landscape of Block-Based AI Education	3
2.2	Transparent Algorithm Learning in Block-Based Environments	6
2.3	Learning-by-Building: Project-Oriented AI in Scratch and Snap!	10
2.4	Block-Based Computer Vision in Practice: Pose, Gestures, and Recognition	13
2.5	Pipeline-Oriented Visual Tools Covering End-To-End ML workflows	16
2.6	Scaffolding, Feedback, and Extensibility for Block-Based AI Learning . .	19
2.7	Deep Learning Without Text Code: Neural Networks as Blocks	23
2.8	Research Gaps and Motivation for VisionBlocks	24
3	Methodology	27
3.1	System Overview	27
3.2	System Architecture	31
3.2.1	Client Layer	32
3.2.2	Application Layer	32
3.2.3	Data Layer	33
3.3	Module-Based Learning Structure	34
3.4	Module 1: Dataset Exploration	34
3.5	Module 2: Image Preprocessing	36
3.5.1	Stage 1: Grayscale Conversion	37
3.5.2	Stage 2: Brightness, Contrast, Blur, and Sharpen	38
3.5.3	Stage 3: Resize and Pad	38
3.5.4	Stage 4: Normalization	38
3.5.5	Stage 5: Loop and Export	39
3.5.6	Bonus Stage: Edge Detection	39
3.5.7	Summary of Learning Design in Module 2	39
3.6	Module 3: Model Building and Training	40
3.6.1	Stage 1: Train/Test Split	41
3.6.2	Stage 2: Creating a CNN Model	41
3.6.3	Stage 3: Training of The Model	42
3.6.4	Stage 4: Evaluate & Predict	42
3.6.5	Summary of Learning Design in Module 3	42

4	Experimental Design	45
4.1	Participants	45
4.2	Experimental Conditions	45
4.3	Experimental Procedure	46
4.4	Focus Group Feedback and System Refinement	46
4.5	Measured Outcomes	47
5	Test and Results	49
5.1	Learning Gain	49
5.2	Engagement	51
5.3	System Usability	52
6	Conclusion and Future Work	55
	Appendix	57
	List of Figures	59
	References	62

Chapter 1

Introduction

Computer vision is a part of artificial intelligence that specifically focuses on helping computers understand visual inputs, such as images and videos, in order to extract some meaning from them that can be used in certain tasks. It is the backbone of many state-of-the-art solutions, such as healthcare applications, autonomous vehicles, robotics, and multimedia systems. The increasing adoption of these systems has led to a rising need to teach computer vision in educational institutes from a fairly early stage in higher education. The problem with computer vision is that learning computer vision is difficult for a newcomer because computer vision deals with many different components, such as data, image processing, and learning, together in a single system.

In recent years, block-based programming systems have been used more and more as a learning tool for programming and Artificial Intelligence. Well-known examples of such systems include Scratch and Snap!, as well as a wide range of block-based extensions for teaching artificial intelligence and machine learning concepts [13, 9, 14, 6]. Block-based programming systems are where students use visual blocks that convey computational statements. This is as opposed to programming, which uses text statements. The use of block-based programming systems has been shown to increase learning because the blocks are easier to understand than plain text, making abstract concepts more accessible to learners. Additionally, block-based programming systems can increase learning motivations, especially when learning programming.

Despite these advantages, many existing block-based systems for artificial intelligence place their primary emphasis on model training and experimentation. In computer vision contexts, learners are often introduced directly to classifiers, frequently neural networks, while the earlier stages of the workflow receive comparatively less attention. Several platforms are designed to support rapid model creation and immediate experimentation [26, 20, 6], which can be motivating, but this emphasis can unintentionally obscure the role of the preparatory steps that precede learning.

In practice, a significant portion of a computer vision pipeline is taken up by pre-processing tasks like resizing images, adjusting the brightness or contrast, or normalizing pixel values. These steps are necessary to ensure consistent inputs and reliable model

behavior. However, in many educational tools and curricula, preprocessing is often seen as a technical step that must be done before "the real work" of modeling can start. This can lead learners to follow preprocessing steps without fully understanding their rationale or impact on model behavior.

To fill this gap, this thesis presents VisionBlocks, a block-based interactive learning environment created to help beginners see and understand how a computer vision pipeline works. VisionBlocks is designed for first-year university students with little or no prior experience in computer vision and is aiming to help them build a clear mental picture of how computer vision systems work. VisionBlocks doesn't hide preprocessing or make it just a step in the process. Instead, it makes it the focus of the learning experience, encouraging students to think about how their choices about preprocessing affect the results of modeling. While many current educational platforms try to support clear and complete machine learning workflows, preprocessing is rarely seen as a main learning goal on its own [23, 26, 22].

This thesis is divided into the following chapters. Chapter 2 provides an overview of relevant work on block-based artificial intelligence education. Chapter 3 introduces VisionBlocks and describes its system design in detail. Chapter 4 explains the experimental method and design used to evaluate VisionBlocks. Chapter 5 presents the tests conducted and the results obtained from the experiment. Chapter 6 concludes the thesis and discusses its limitations and directions for future work.

Chapter 2

Literature review

2.1 Mapping the Landscape of Block-Based AI Education

A brief review of block-based and visual tools for AI education is offered by Reham and Nada, suggesting that “learning-by-doing environments” with broad accessibility could contribute to all learners building meaningful AI knowledge whatever their backgrounds. A survey and review of specific AI/ML learning tools is undertaken, and, in addition, a division of this learning-technologies space into three topics is made: (1) block-based platforms to support model-building through reduced workflows for AI/ML modellers, (2) block-based implementation of AI/ML algorithms designed for increased AI “black box” system understanding, and (3) add-ons to improve and enrich Scratch or Snap! environments. A key part of this is also to provide this helpful summary table comparing and contrasting environments to succeed in AI-ML learning such as: “Snap! AI (eCraft2Learn), ML for Students, Teachable Machine, Google Visual Blocks, Cognimates, Milo, and LearningML with regards to programming mode, target age, supported modality, capacity to train with personal data, model export, real-time feedback availability, robotics/device support, and the degree to which AI internals are made visible to learners.” Several summary views of research work are offered: DBR activity for Scratch-based algorithm exposure (using K-means and NN), SnAIp’s code for “reinforcement learning (Q-learning), and visualization of the ‘Q-Table,’” gesture recognition system integration of “hardware and software workflows (for example, Scratch Nodes ML),” and “supervised learning add-ons (for example, Naive Bayes Scratch extension),” and projects to “enlarge” and assist “learning-by-doing (for example, Scratch-text language bridge).” In this final summary, it is suggested in this review of AI-ML learning environments, while most of these are interesting and capable of turning abstract AI/ML learning into fun and useful environments for all learners without concern for start-out background, the available research evidence supporting these tools is often limited in scale or depth, and many environments remain constrained to a small set of predefined models, with little flexibility to incorporate alternative paradigms or external model APIs [4].

Batni et al. conducted a systematic literature review of the literature regarding the use of Scratch as an educational platform in K-12 schooling that is informed by the steadily increasing use of block programming editors and the concurrent discussion regarding the pinpointable educational efficacy of block programming instruction. Following the PRISMA checklist, the authors conducted a search using Scopus and Web of Science, set up precise inclusion/exclusion parameters (eng., journal articles, use of Scratch within the classroom setting of K-12 schooling, empirical articles, published during the period from 2018 to 2022, excluding conference proceedings and non-empirical articles), and narrowed down an initial list of 1306 articles to the final set of 27 articles that met inclusion criteria. Organized to address the response to four specific research queries (distribution of the articles, prevailing methodological norms, typical non-technology topics, and an exploration of the use of Scratch outside of technology/CS studies) is an exploration of published trends that indicate the highest number of articles published during the period ending in 2020 (33.3%) and again in 2021 (29.6%), predominantly from Turkey, Serbia, and China, consisting of four articles each, but also represented from other, unnamed nations to a lesser degree. In regard to methodology, the two most prevalent types of designs were those considering mixed methods designs, consisting of 11 articles, while the second most typical methodological categorization included qualitative articles, consisting of 10 articles, and thirdly, articles that categorized methodologically as being strictly quantitative, consisting of six articles, with additional considerations explored within testing, examination of Scratch project productions/code, and classroom observations, consisting of nine, eight, and six articles, respectively, alongside the use of interviews and surveys. In regard to the educational focus, the authors indicate that “17 articles use Scratch to support educational topics related to CS/ICT, while 10 articles use Scratch to support other subjects, mainly STEM/STEAM topics, particularly math, but also other subjects, such as English, and one workshop on AI topics.” Of the articles reviewed, the authors discuss the typical positive educational outcome regarding the use of Scratch, and specify that despite the largely positive finds, there is still limited long-term evidence, many of the studies have smaller sample sizes, the shift from block-based to text-based programming is still not well understood, and the skill of creativity, or collaboration is still not well understood, using traditional measurement constructs [5].

Yim and Su conducted a scoping review of empirical research involving learning tools about AI literacy in K-12, influenced by the proliferation of AI education programs and the uncertainty about the appropriateness and relevance of tools, techniques, and learning outcomes across different levels. After a PRISMA-guided search across multiple databases (ACM, EBSCO, Web of Science, and Scopus), together with snowballing, the researchers find 46 studies conducted across kindergarten, primary, and secondary levels of education, with publications overwhelmingly in the last few years, and studies carried out in different countries. The study aggregates answers to research questions in the areas of learning tools, learning approaches, outcomes, and methodology. In terms of tools, the reviewed studies can be grouped into intelligent-agent style platforms (e.g., Teachable Machine, LearningML, Machine Learning for Kids), software-focused environments (notably Scratch, Snap, Python), hardware/robotics systems, and a smaller number of unplugged activities. Pedagogically, the dominant pattern is constructivist,

project-based learning, often combined with human-computer interaction, play/game-based activities, and collaborative work. Reported results across the reviewed studies include positive cognitive outcomes (basic understanding of ML principles, rule-based systems, AI limitations, and ethics), affective/behavioral outcomes (motivation, interest, self-efficacy, engagement), and in some cases satisfaction and soft-skill development (e.g., collaboration and communication). Methodologically, research most often applies mixed methods designs, with frequent reliance on survey questionnaires, artifact or performance-based assessments, and interviews, and the review notes uneven methodological rigor and a need for more objective and standardized measures of learning gains [24].

Yim and Su conducted a systematic literature review of artificial intelligence literacy education in primary schools where there is no current evidence-based recommendation on what resources are suitable, what contents should be covered, or how to assess effectiveness. Using a PRISMA process, and based on searches in Scopus and Web of Science, about 25 empirical studies are identified, dating from 2019 until early 2024. The review consolidates typologies on how AI literacy is defined, taught, and assessed in primary education. In all the studies, AI literacy is described as a mix of digital literacy necessary to engage and work with AI, computational thinking, critical data literacy, and AI ethics. The majority of the works are informed by constructionist or constructivist theories, with several studies additionally drawing on the ARCS motivational model. Both project-based learning and programming activities predominate as teaching approaches, and to a lesser extent human-agent interaction, as reflected in the tools used for learning: intelligent agents (e.g., Teachable Machine, robots) constitute the majority, followed by software platforms like Scratch and fewer unplugged activities. Results indicate that reported outcomes across studies are overwhelmingly positive, including improvements in students' conceptual understanding of AI and machine learning, enhanced motivation, engagement, and self-efficacy, and generally high levels of course satisfaction. Nevertheless, the review highlights current issues and recurring themes such as inconsistent curricular standardization, reliance on short-term interventions, inconsistency in measurement rigour, technical or scalability constraints, and gender-related differences [25].

Gresse von Wangenheim et al. conducted a ten-year (2010-2020) systematic mapping study of research output on tools that facilitate teaching machine learning at K-12 levels through the creation of customized ML models, responding to the lack of previous systematic overviews exclusively examining tool-based features. With searches through the major digital libraries (plus Google and snowballing), and using inclusion criteria which specify that tools must be capable of supporting custom ML model creation for K-12 learning, they identify 24 relevant artifacts from 16 different tools. They contrast tools in educational typicalities, ML-platform capabilities, deployment options, and tool development/evaluation. The map also reveals that tools are most frequently used in compact extracurricular units (frequently between 1-4 hours), target ML novices (commonly at the high-school level), and are usually accompanied by tutorials or lesson materials. On the technical level, most tools can be used to design a simplified ML pipeline focused on data collection/labeling, model training (often including transfer learning), and testing, but offer only little support for requirements analysis and very limited

support for feature engineering. Tools are predominantly focused on recognition-based tasks, particularly image recognition, and often work in conjunction with block-based environments—most notably Scratch and App Inventor—to facilitate deployment to games or mobile applications. On outcomes, the paper aggregates evaluation results and notes that feedback to learners is reported in (often small-scale) studies as being of good usability and that this allows students to iteratively test hypotheses about data or models, discussing model behavior and limitations. The authors also point out deficiencies, such as inadequate teacher and assessment support, poor reporting of tool-development processes, and a call for more robust and wide-scale evaluations [23].

2.2 Transparent Algorithm Learning in Block-Based Environments

Estevez et al. describe and pilot-test a teacher-guided Scratch tutorial aimed at an “experiential gentle introduction” to the basics of AI through direct engagement with simplified versions of two machine-learning algorithms: K-means clustering (unsupervised learning) and a simple artificial neural network (supervised learning). The intervention, grounded in both Design-Based Research (DBR) and Experiential Learning Theory (ELT), takes the form of a guided use/modify/create cycle, where students fill out partially completed Scratch programming templates for specific missing code fragments, before running the code and experimenting/modifying the programs to see their behavior and constraints. In the K-means algorithm experience, the focus is on distributing points in a randomly generated 2D data-set to their nearest of K randomly located cluster ‘centers,’ as determined by colored clusters, although the authors make it clear that in the pilot, the programming task simplifies to the assignment step alone, rather than the full cycle of recalculating the cluster centers. In the neural network experience, the algorithm simulated uses a simplified ‘single-layer’ perceptron model trained on an ‘AND’ logic gate and subsequently an ‘OR’ gate, where derivatives required for full backpropagation are beyond the students’ mathematical curriculum, and weight updates are performed using a simplified error-based update rule with a threshold activation function, allowing weight and error changes to be made visible during the iterative optimization phase. In the pilot, the intervention took place in a single extra-curricular session with 37 students aged 16-17, 40% of whom were girls and 60% boys, all of whom had two classroom sessions of prior Scratch experience, working in pairs, and taking around 40 minutes to complete after a brief introduction to AI fundamentals. The intervention impact was assessed using a two-part questionnaire designed to address how computational experience alters perceptions of the societal effects of basic AI algorithms, consisting of two free-response questions and four Likert-scale items measured on a five-point agreement scale. Findings indicate a shift in students’ free-response descriptions of AI from vague narratives centered on danger and destruction to conceptions emphasizing learning, prediction, algorithms, and data, alongside a corresponding change from fears of hostile robots to concerns about data misuse and privacy violations. Quantitative results show statistically significant

pre/post changes on items related to perceived difficulty of understanding AI, detection of AI hype, and “Terminator-like” scenarios, with large effect sizes and very small p-values, while responses concerning legal regulation changed little overall, though a shift from indifference to agreement was observed. The paper concludes that Scratch-based activities using template completion can effectively support experimentation with basic AI mechanisms, while noting limitations including the small sample size and the single-run implementation with limited time for extended scaffolding practices [9].

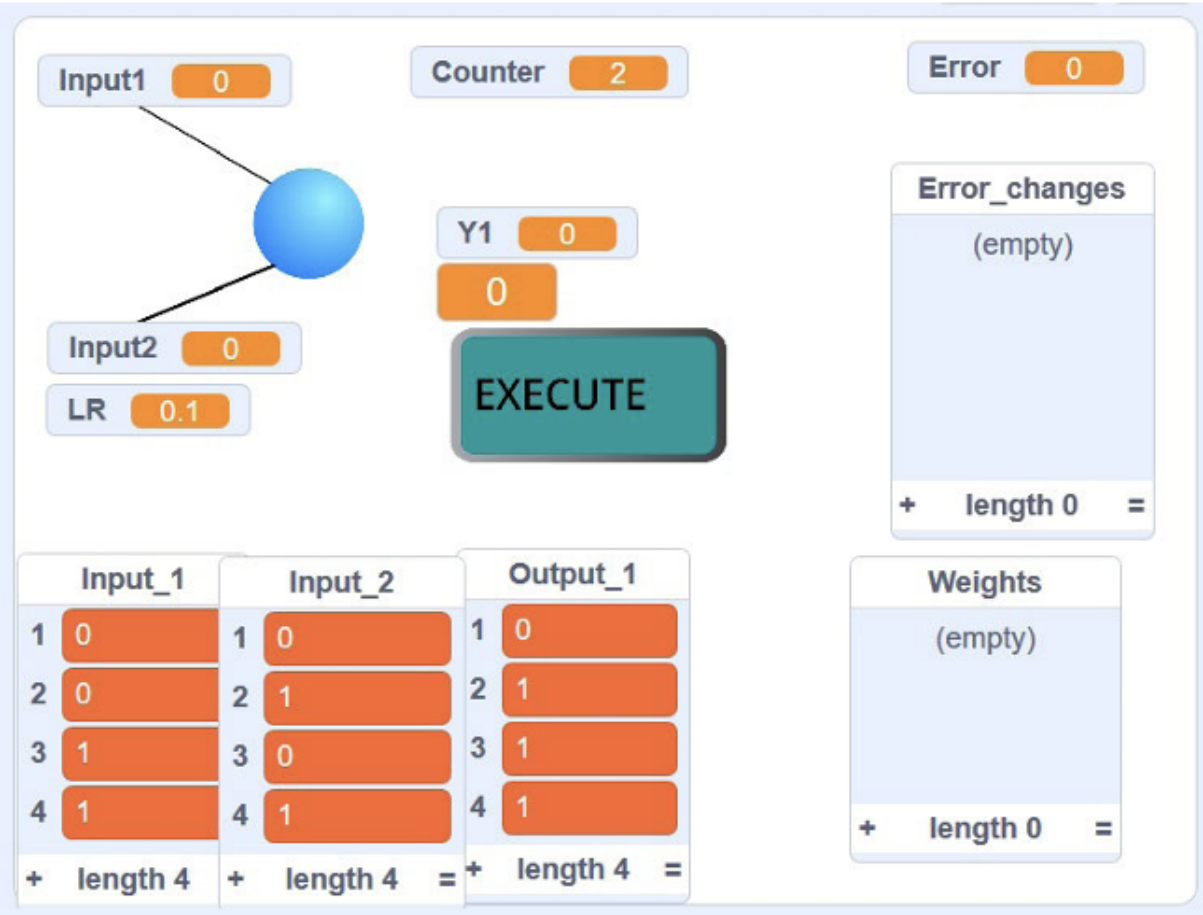


Figure 2.1: Illustrative excerpt from Estevez et al.

Jatzlau et al. discuss a persistent weakness of many block-based machine-learning learning tools: while their use makes the interaction with computation more accessible, they regularly hide the actual learning process because model training and inference is outsourced to an API or server, which learners cannot inspect nor reason about how algorithms actually work. In order to respond to this “black-box” issue, the authors put forward four design principles for block-based ML education: making learning mechanisms visible by allowing learners to inspect and manipulate the algorithmic elements, keeping all computation inside the programming environment by not relying on the use of servers or external code, focusing on meaningful and practical projects instead of isolated instruc-

tional exercises, and treating trained models as material artifacts that can be compared, discussed, and reflected upon. These principles are instantiated in SnAIp, a Snap!-based educational project whose first component focuses on reinforcement learning. The authors have implemented a complete Q-learning algorithm entirely within Snap!, avoiding JavaScript wrappers or API calls, so that all state transitions, rewards, and updates remain observable to learners. The reason for using reinforcement learning here is that it can be expressed using the simple Q-table representation and provides visibly changed behaviors within a small state space in relatively less time, hence suitable for classroom use. This paper proposes a structured learning sequence targeted at secondary-school students (about 14+ years). The activity starts with an unplugged Hexapawn, where the students can play against each other to understand the concept of the reinforcement learning loop. Then, there is the Banana Hunt activity in Snap!, where the learners solve a template-based puzzle by sequencing predefined agent blocks in order to understand how an agent learns from reward. Later stages introduce a "Pimp My Game" activity using coding cards to support learner-driven game modification using reinforcement learning, and then a "Behind the Scenes" phase that exposes the evolving Q-table in order to make explicit the dynamics of learning. This paper thereby illustrates how reinforcement learning can be taught transparently and constructively in a block-based environment and offers a complement to API-driven ML tools, which hide algorithmic processes [11].

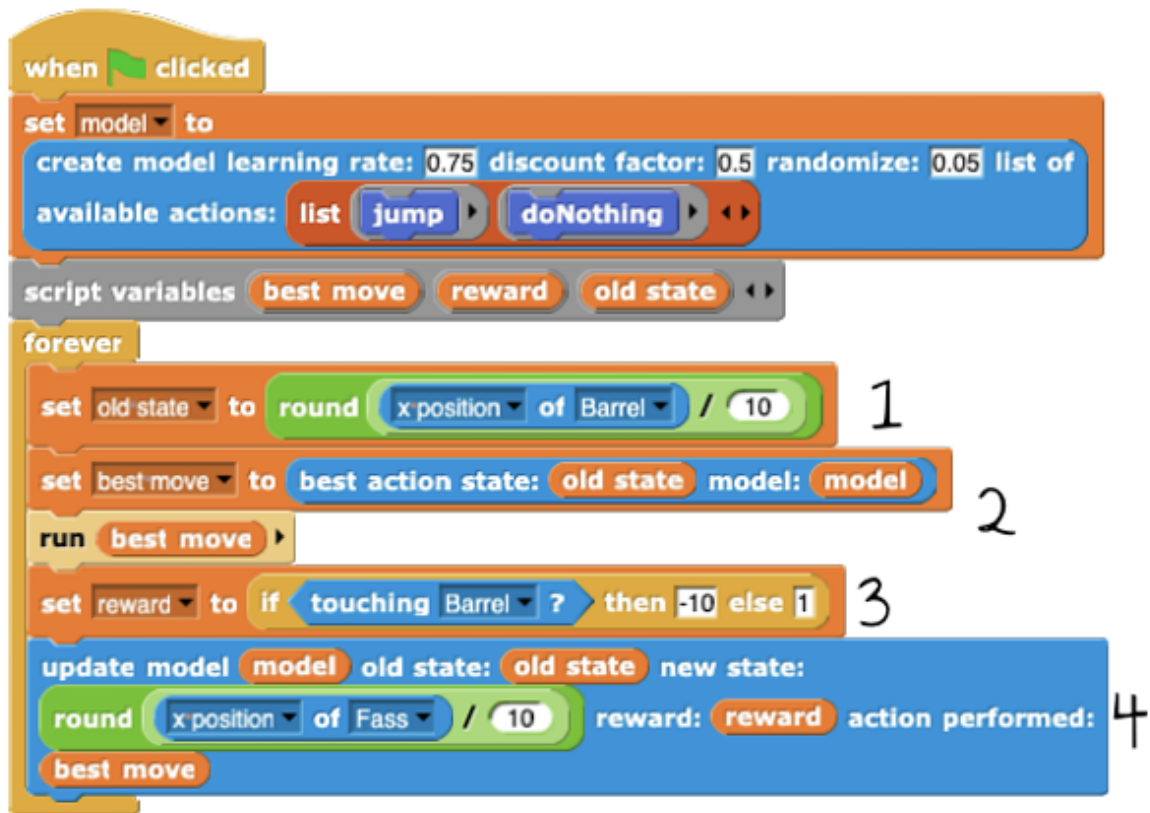


Figure 2.2: Illustrative excerpt from Jatzlau et al.

Quiroz and Gutierrez introduce Scratch-NB, a Scratch extension designed to help very young learners (K-8) build a simple Naive Bayes text classifier while still seeing key parts of the supervised learning pipeline, rather than treating the model as an invisible black box. The extension adds blocks that map directly to the algorithm’s core steps, including initializing a classifier, training categories with labeled examples, computing category probabilities for a new input, and selecting the most likely category. The authors also position Scratch-NB against tools like Teachable Machine and Machine Learning for Kids, arguing that many popular tools simplify ML by outsourcing computation and hiding the algorithm’s internal structure. They report an informal university-based workshop with 40 children aged 10-12 who self-declared no prior programming or AI experience. Participants were split into two groups, one using Scratch-NB and one using ML4Kids, and both groups built a small emotion classifier. Evidence came from tutor observations, constructive interaction sessions, verbal probing questions, and (for Scratch-NB) analysis of the Scratch programs students created. Results suggest strong acceptance and engagement for Scratch-NB, with most learners able to assemble a working classifier, and about one third able to clearly explain the ML stages and relate them to the blocks. When asked to choose, 14 of 20 preferred Scratch-NB over ML4Kids, often describing it as more playful and more informative about how the model works [19].

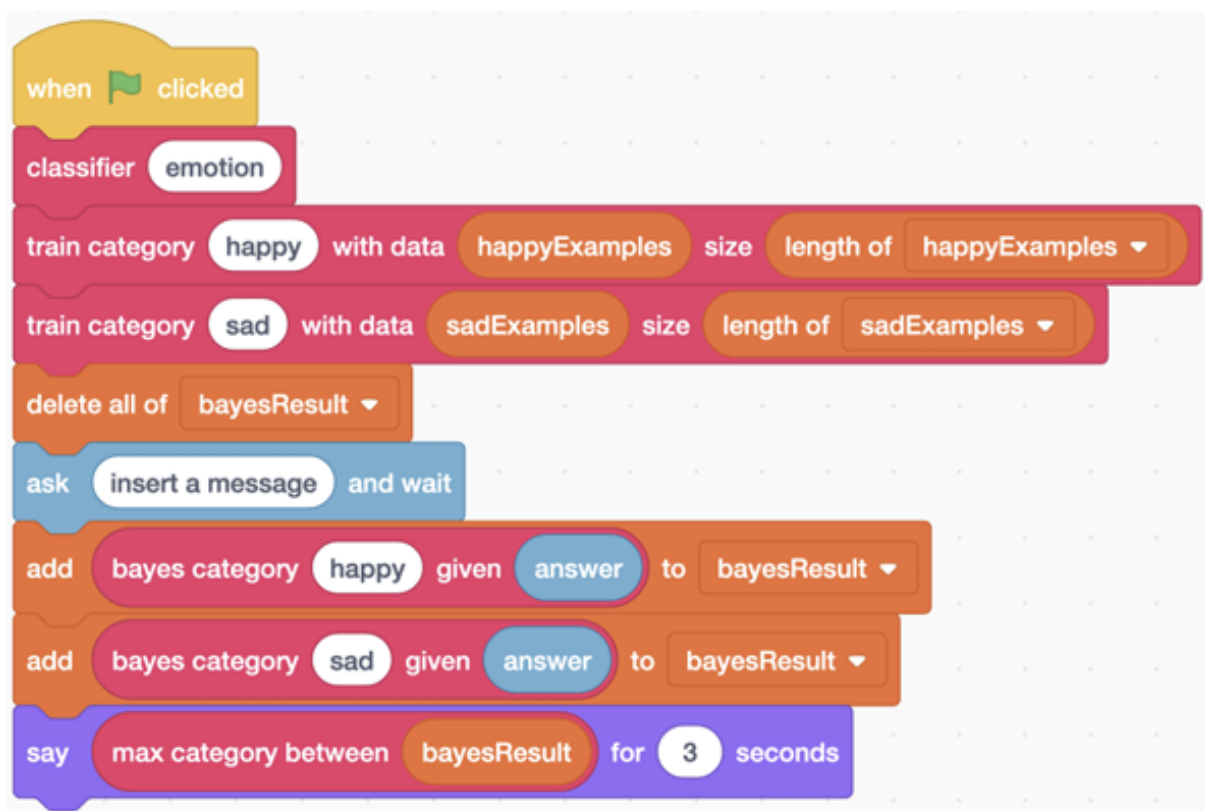


Figure 2.3: Illustrative excerpt from Quiroz and Gutierrez.

Estevez et al. describe a Scratch-based workshop designed to demystify AI by letting

learners program simplified versions of classic algorithms and see the “rules” behind intelligent behavior. Although the title emphasizes undergraduates, the proposed activity is explicitly written for students aged 16 to 18, and the paper focuses on how teachers can run the session rather than reporting a formal classroom study. The workshop includes three hands-on exercises. First, students complete a partially prepared Scratch project that performs K-means style clustering by computing distances from points to randomly initialized centers and then coloring points by the nearest center. Next, students build and train a simple single-neuron network on AND and OR logic gates, and watch how weights and error change over iterations. Finally, they extend this to a small multi-layer network with three inputs and a hidden layer, again trained on AND and OR, by filling in missing computations and update steps. The teaching approach is structured and teacher-guided, with short introductions, pair work, and “fill the gaps” code completion so students focus on the underlying math and logic instead of writing everything from scratch. In terms of results, the paper does not provide learning outcome data; instead, it offers the completed designs, screenshots, and equations as an implementable proposal, and it frames evaluation as future work to be measured in repeated deployments [10].

2.3 Learning-by-Building: Project-Oriented AI in Scratch and Snap!

Kahn et al. share results from an exploratory workshop carried out among 40 senior high school and vocational school students aged 16-17 from Indonesia on the topic of artificial intelligence programming using the Snap! graphics-based programming software specially extended by the eCraft2Learn project. The tutorial design uses a “learning by construction” strategy based on a notion of AI as agent systems and focuses on helping the learners conceptually master the notion of the agent’s environment, agent perception, and agent action. Two contrasting topics were covered: speech synthesis, during which the learners were enabled to vary parameters like pitch, rate, language, and pronunciation ambiguity, and image machine learning, during which the learners implemented and evaluated webcam classifiers to identify gestures and outcomes of a “rock-paper-scissors” game. Post-workshop essay and classroom observations were the instruments of choice to assess the results of the tutorial. Conclusion was that while a general concept understanding and high enjoyment and motivation levels were attained by the vast majority of the participants, the notion of environmental uncertainty was understood less consistently, especially by the uninitiated programming group. Another aspect underlined by the research was that several real-world challenges emerged when attempting to conduct AI education in a developing-country context, including heterogeneous computer architectures, limited internet connectivity, and execution slowdowns, which at times disrupted the workshop flow and learners’ progress. As a general conclusion on this specific research, the authors highlighted the effectiveness of graphics-based programming software as an introductory tool for AI education, while emphasizing the need for clearer instructional guidance and more infrastructure-resilient software designs to better support learning in

such contexts [13].

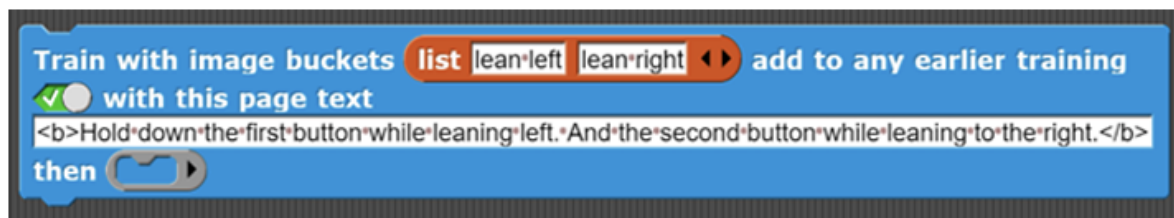


Figure 2.4: Illustrative excerpt from Kahn et al.

Kahn et al. describe a library of AI blocks integrated into Snap! that lets learners build programs using speech, vision, language, and neural networks without leaving a block-based environment. The paper is written as a systems and demonstration overview, explaining what the blocks do, how they were designed, and how they have been used in educational settings. The library includes simple and “expert” versions of key features so beginners can start quickly while advanced users can control options. For speech, the blocks support synthesis and recognition using built-in browser services, including background listening patterns. For computer vision, the authors describe both cloud-service blocks (Google and Microsoft) and a shift toward on-device TensorFlow.js models for tasks like image classification, object detection, segmentation, and pose detection, which avoids sending data to servers and can reduce latency. The library also supports word embeddings in multiple languages, plus english sentence embedding and question answering, and provides blocks for creating and training neural networks directly in Snap!, including hyperparameter optimization. Alongside the blocks, the paper highlights an interactive web-based programming guide and over 30 sample projects designed to be easy for students to modify. In terms of results, the evidence is descriptive rather than experimental. The authors report that the blocks have been used by students as young as six and in workshops across several regions, and they share observations from an India pilot where children created projects like mask detection and style transfer, with teachers noting students could explore AI in fun ways and take more autonomy in learning [14].

Kahn and Winters describe how they extended Snap! with child-friendly blocks that let students access powerful AI services, without needing to learn complex professional APIs. The paper motivates this by noting that speech synthesis, speech recognition, image recognition, text analysis, and machine learning services are widely available from major providers, but are normally too technical for school learners to use directly. Rather than creating a new environment, the authors focus on enhancing Snap! because it is Scratch-like but more expressive, runs in the browser, and can be extended with JavaScript. The paper explains three main additions. First, speech synthesis blocks built on browser speech APIs, with an easy version and a more advanced version that exposes options like pitch, rate, volume, language, and voice. Second, speech recognition blocks, including a continuation-based design, plus an alternative that hides the complexity by broadcasting events and storing the last utterance in a global variable. Third, image

recognition blocks that can call Google, IBM, or Microsoft services, plus camera setup support. Results are early and descriptive. The authors report preliminary testing with about 25 undergraduates and 6 children (ages 7 to 13), where users generally understood the blocks quickly and enjoyed adding speech to projects. Feedback also revealed design needs, like handling silence and giving clearer responses when recognition fails [15].



Figure 2.5: Illustrative excerpt from Kahn and Winters.

Druga’s Growing Up With AI: Cognimates presents the design and early evaluation of Cognimates, a child-centered platform that extends a Scratch-like visual programming environment with AI capabilities, aiming to help children (roughly ages 7–14) both program and teach interactive agents using cognitive-service extensions and simple model-training workflows. The thesis documents the path from early Scratch extensions for robotics and vision to a more unified platform, and it contributes learning guides, starter projects, platform design guidelines, and proposed evaluation metrics for children’s interaction, understanding, and perceptions of AI technologies. Empirically, the thesis reports multi-session studies in informal learning settings and compares them with shorter international sessions that follow a similar protocol: children first interact with smart agents, then program them, and complete perception questionnaires and interviews. In one detailed site report, children were highly engaged and collaborated well, but novices needed time on basic programming structures and some struggled with reading and typing, slowing progress. After programming and training, many children shifted answers toward “maybe”; across studies, they often became more skeptical about whether agents have feelings, while changes in perceptions of intelligence, memory, and overall “smartness” varied by context. The thesis also notes practical friction such as language-specific speech recognition behavior, camera-extension loading issues, and slow Wi-Fi, motivating more robust deployment ideas [7].

2.4 Block-Based Computer Vision in Practice: Pose, Gestures, and Recognition

Agassi et al. introduce Scratch Nodes ML, a playful system that lets children build their own gesture recognition classifiers and then use them inside Scratch projects as new blocks. The paper starts from a simple problem: many everyday machine learning systems are hidden behind “black boxes,” so children may form inaccurate mental models unless they get direct experience with the learning process. Scratch Nodes ML makes selected parts of the pipeline visible by having children create gesture classes, record labeled examples, test the classifier on new gestures, and then generate Scratch event blocks that trigger when a gesture is recognized. The system combines a physical Scratch Node device with an accelerometer and feedback components (LEDs, speaker), a tablet running an extended Android version of Scratch, and a Scratch 3.0 extension that provides a gesture editor similar to Scratch’s sound editor. For classification, the paper uses a supervised approach based on a 1NN with Dynamic Time Warping to compare live gesture streams against the stored examples, with predictions shown in real time and changing as new samples arrive. A key “result” the paper emphasizes is the interaction pattern it enables: children can iteratively improve recognition by collecting more examples, and they can also observe that classifiers often become gesture-specific to the person who provided the training data. The paper illustrates this through a detailed usage scenario (Harry Potter “spell” gestures) that shows trial and error, collaborative roles, and the way data quality affects accuracy [1].

Jordan et al. present PoseBlocks, a Scratch-based toolkit that helps students build AI projects driven by natural body interaction, like the kinds of systems they already see in AR filters and motion-based games. The toolkit runs in a web browser and adds new blocks for hand, body, and face tracking, plus facial expression and emotion recognition, and it also lets students import their own custom image, pose, or audio classifiers trained in Teachable Machine. The authors deliberately design the blocks to feel “tinkerable,” for example by offering blocks that directly move sprites to tracked body parts so students can see immediate effects without building complex logic first. To support classroom use, the editor includes a video recorder, sample projects, PoseBlock Cards (like Scratch Cards), and an ethical design activity using an Ethical Matrix. A key technical choice is privacy and responsiveness: models run locally in-browser, and the toolkit can use precomputed video loops with cached model outputs for smoother iteration on slower devices. They report pilots of a week-long “Dancing with AI” curriculum delivered online to 46 students aged 9 to 14, with 34 in the research portion and 21 completing the full week. Looking at final projects, they found students created a wide range of applications, most using the AI blocks, including games, wellbeing ideas, educational tools, and emotion-driven interactions. Teacher feedback describes high engagement and strong creativity, and the authors note practical limitations around webcam access conflicts on Windows and camera support issues on iOS, which required workarounds [12].

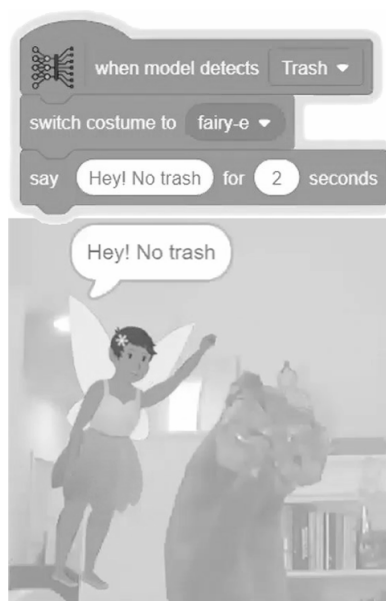


Figure 2.6: Illustrative excerpt from Jordan et al.

Carney et al. present Teachable Machine, a web-based interface that helps people train their own classification models without coding, using images, sound, or pose as input. The paper frames the tool as a response to digital inequality, since many people are affected by machine learning but cannot participate in creating models. Teachable Machine is designed for classrooms, rapid prototyping, and accessibility contexts, and it relies on transfer learning so users can train useful models quickly from small personal datasets. The interface walks users through an end-to-end workflow laid out left to right: defining classes, collecting examples (from webcam, mic, or uploaded files), training, testing, then optionally exporting the model for use elsewhere. The paper also explains several “under the hood” choices meant to support safe experimentation and learning. Models train on-device in the browser, so user data does not need to leave the computer, and users can download the trained model via TensorFlow.js. For users who want more control, an advanced panel exposes basic hyperparameters like epochs, batch size, and learning rate, and evaluation views display training and validation curves, holding out a portion of examples for validation. Results are reported as adoption and use in the wild rather than controlled learning studies. The authors state that people have trained over 125,000 models, and they describe examples of educators using Teachable Machine to teach supervised learning and bias, students using it for creative course projects, community-created tutorials in multiple languages, and accessibility prototyping such as bespoke interfaces for people with ALS [6].

2.4. BLOCK-BASED COMPUTER VISION IN PRACTICE: POSE, GESTURES, AND RECOGNITION

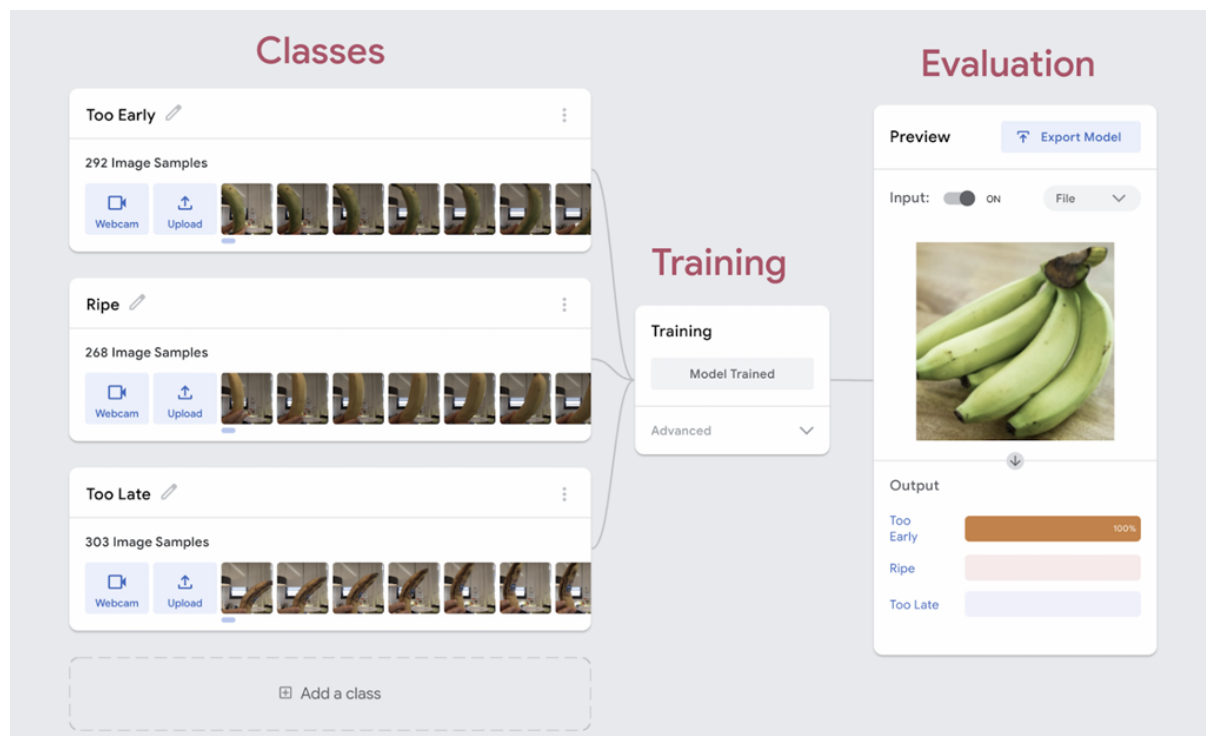


Figure 2.7: Illustrative excerpt from Carney et al.

Pacheco et al. report a small qualitative case study that uses Machine Learning for Kids (ML4K) to explore how an introductory machine learning activity can support computational thinking (CT) in school. The study takes place with a 9th grade class in a public school in São Miguel (Azores, Portugal), focusing on 10 students (aged 14-16) who already had some exposure to block-based programming. Students work in pairs for a 90 minute session built around the familiar Rock, Paper, Scissors game. They first receive a brief introduction to AI and machine learning, then create an image recognition model in ML4K by collecting webcam examples of the three hand shapes, training the model, testing it, and importing it into ML4K's Scratch 3 environment. They use and slightly modify an existing ML4K game template so the trained model drives the gameplay. Data collection is based on structured pre and post interviews (about 10 minutes each), which ask students to explain AI, list ML examples, and describe CT ideas like abstraction, decomposition, algorithms, and pattern recognition. Results are reported as clear shifts in interview responses. Before the activity, only 10% said they knew machine learning and only one student could give an ML example. Afterward, all students reported knowing machine learning and computational thinking, and more students could name CT pillars and provide ML examples, especially recognition tasks. The authors conclude that ML4K can be a useful way to connect AI activities to CT development [16].

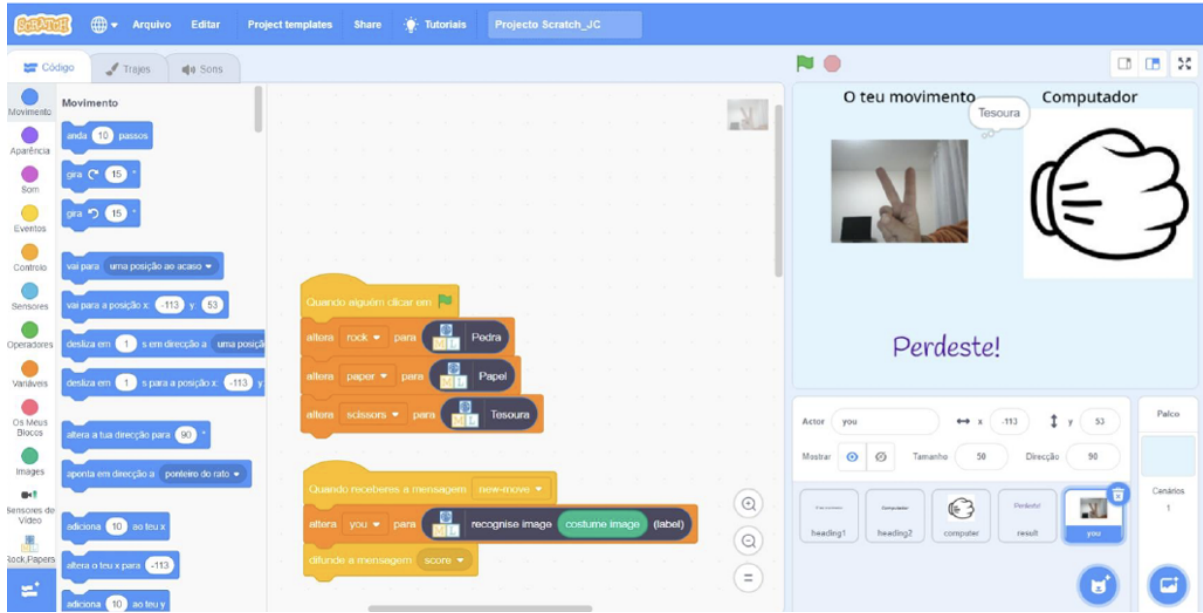


Figure 2.8: Illustrative excerpt from Pacheco et al.

2.5 Pipeline-Oriented Visual Tools Covering End-To-End ML workflows

Zhou proposes AIBEE, a cloud-hosted block-coding environment intended to make AI learning feel more like building with Scratch, but with blocks that cover the full AI modeling process from data importing and processing to model training and testing. The paper argues that existing block-based AI tools often feel like “black boxes” (they show outcomes but not components), are hard to build curricula around, and do not support exporting models for real use. To address this, AIBEE is designed around functional block categories for datasets, preprocessing, models, training, visualization, evaluation, and running, so learners can drag blocks into a central workspace and immediately see results without writing code. The interface is a three-panel layout: a block palette and teaching zone on the left (including a chatbot and an “AI instructor” that initially is a prerecorded video), a build space in the center, and an output panel on the right that behaves like a notebook view. AIBEE also includes a code view that exposes the generated Python. Technically, AIBEE is implemented as a cloud-based web application that combines a browser-based block editor with backend services for storing data and running models. While students interact only with blocks, model training and execution are handled through a Jupyter environment hosted on AWS SageMaker, allowing results to be displayed immediately without local setup. Regarding results, the paper does not yet report classroom learning outcomes. Instead, it describes a planned comparative evaluation consisting of free five-day summer camps with approximately sixty high-school students, who will be assigned to use AIBEE, Scratch, or App Inventor. Student learning and experience are intended to be assessed through post-workshop surveys and quizzes [26].

Rodríguez-García et al. present LearningML, a free, web-based platform designed to teach supervised machine learning through hands-on projects while also supporting computational thinking. The authors argue that common classroom tools (for example Scratch plus external ML services) often introduce practical friction such as registrations, API keys, and unpredictable cloud latency, and they can keep model building too opaque. LearningML is built around a clear four-step workflow that learners can iterate through: training (collecting and labeling examples), learning (training a model locally in the browser with TensorFlow.js, using an ANN and transfer learning for some modalities), evaluation (testing the model and seeing confidence, and in a planned advanced mode, a confusion matrix), and exporting (moving the model into a programming environment). To make the uncertainty of ML feel concrete, the platform is designed so that a beginner mode can present confidence as natural-language phrases rather than only numbers, although this mode was not yet available at the time of the reported study. The platform connects to a Scratch 3.0 fork that adds blocks for classifying text and images and for adding new examples. For results, the paper reports two pilot workshops with 14 students (around 20 years old) who had programming experience but no AI background. Pre/post testing showed statistically significant gains on both Likert-style knowledge items and multiple-choice questions, with the authors reporting moderate to large effect sizes [22].

Rao et al. present Milo, a browser-based visual programming environment aimed at making data science and machine learning approachable for learners who do not already know a programming language. Instead of notebooks or text code, Milo uses Blockly-style blocks to represent common data science operations, statistics, and ML algorithms, while keeping the workflow in one place. The interface is designed like a single-window IDE, with a block workspace, an output panel, a built-in data explorer that looks like a spreadsheet for inspecting datasets, and a tab that shows the JavaScript code generated from the blocks. A key design choice is that Milo is not event-driven like Scratch. It runs blocks sequentially, top to bottom, which the authors argue makes later transition to text programming more intuitive. Milo also tries to reduce the “black box” feel of ML by pairing algorithm blocks with animated and interactive visualizations, such as showing decision boundaries changing during training or letting users place a test point in a KNN plot and see which neighbors were used. For results, the paper reports a preliminary study with 20 undergraduate CS students in an introductory ML course, used in a flipped-classroom activity on K-means clustering with the Iris dataset. Survey responses were positive: 90% said visualizations were very easy to create and helped their understanding, and 70% felt Milo would be very useful for novice learners [20].

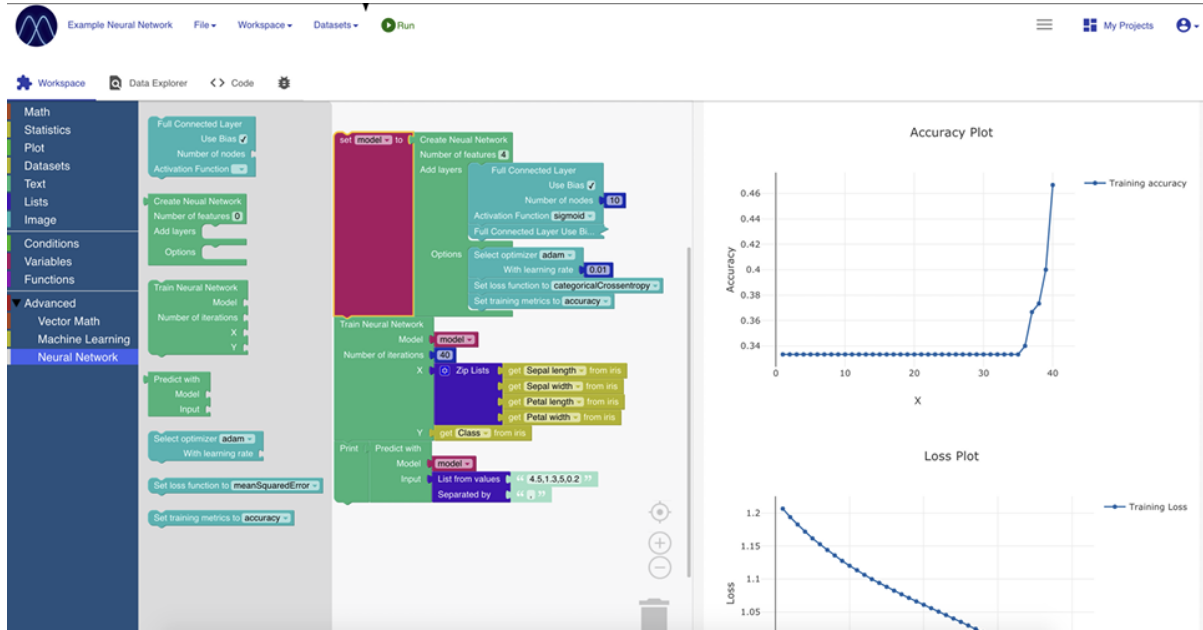


Figure 2.9: Illustrative excerpt from Rao et al.

Du et al. introduce Rapsai, a visual programming platform built to help ML practitioners prototype and evaluate real time multimedia ML applications without constantly switching between scripts, demos, and ad hoc tooling. The paper begins with a formative study of seven practitioners and, together with follow-up brainstorming sessions, uses their pain points to shape six concrete design goals, like making it easy to test models on messy “in the wild” inputs, interactively stress test robustness with data augmentation, compare model outputs side by side, and share pipelines with collaborators quickly. Rapsai’s interface centers on a node graph editor with a library of nodes for input (images, video, audio, live camera and mic, remote streams), effects (interactive augmentation and GPU shader processing), models (pretrained options and custom model runners), and outputs (viewers and comparison tools). A key idea is that most operations run with GPU acceleration where possible, so users can tweak inputs and immediately see how model behavior changes. For results, they evaluate Rapsai through four case studies with 15 ML researchers and engineers working on portrait and scene depth estimation, matting, and audio denoising. Participants could build working pipelines quickly after a short tutorial, and they reported that Rapsai helped them notice error patterns, uncover edge cases, and make more confident model comparisons using qualitative examples. In an exit survey, participants rated Rapsai as more transparent and collaborative than Colab, while still seeing Colab as better for deep control and complex custom code [8].

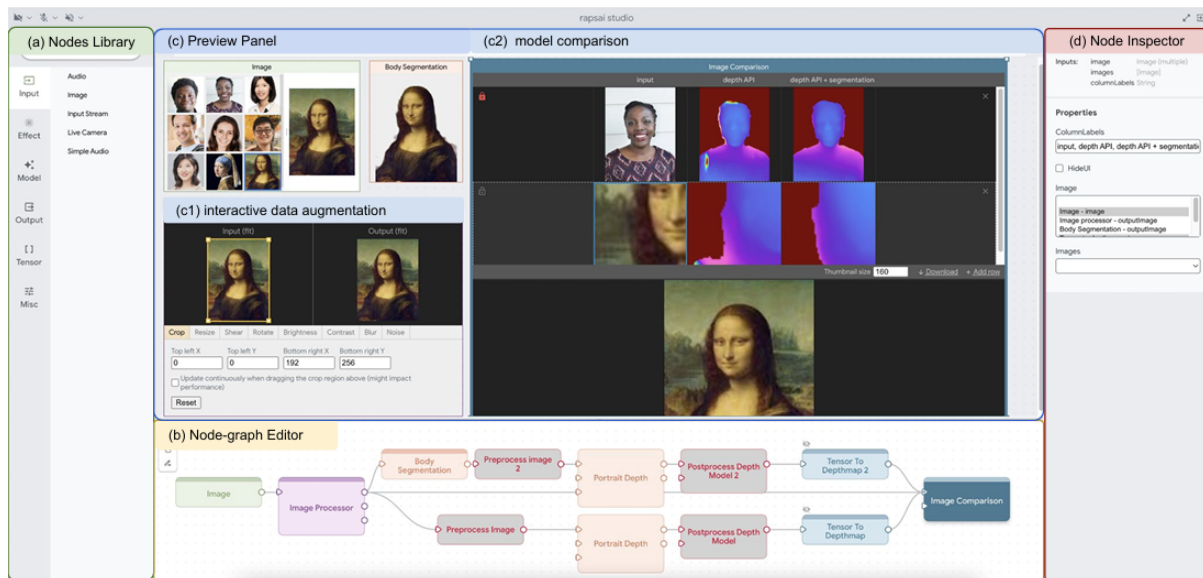


Figure 2.10: Illustrative excerpt from Du et al.

Park and Shin propose a block-based interactive programming environment aimed at solving a specific limitation of many K-12 ML tools: they typically only support training on small datasets, which can weaken students’ sense of why “more data” matters in machine learning. Building on their earlier Tooee idea of conversational Scratch blocks that communicate with a server, they redesign the block interfaces and backend so students can experience both instant training on single samples and large-scale training on datasets of 10,000+ items. The environment supports supervised learning for text and images. Rather than retraining a heavy neural network each time a student adds data, the backend uses a pre-trained embedding-based model (described as Sentence-BERT-based) to convert examples into vector representations and then performs classification by comparing similarities, which keeps interactions fast while scaling to many examples. To show what this looks like in practice, the paper presents two example Scratch-style applications: a question-answering chatbot trained on 11,822 text pairs across 7,784 classes, and a celebrity look-alike classifier trained on 4,431 images across seven classes. For results, they run four short crash courses with 36 teachers and pre-service teachers who build the example apps and complete pre- and post-surveys. Ratings increased after the course on perceived student interest and ease of use for both lower and upper grade levels, with statistically significant gains on three of four items, and comments commonly described the approach as easy and fun [18].

2.6 Scaffolding, Feedback, and Extensibility for Block-Based AI Learning

Reddy, Williams, and Breazeal present LevelUp, an automatic assessment and feedback tool embedded inside a block-based programming platform to support AI education

through supervised text classification projects. The paper’s motivation is practical: many AI curricula use open-ended projects, but teachers often lack scalable ways to evaluate these projects or give students timely feedback while they work. LevelUp addresses this by analyzing both the students’s training data and program structure, then showing a live progress score with checklisted suggestions such as adding more class labels, providing enough examples per label, balancing classes, using multiple text-classification blocks, and using embedded conditionals. They evaluate LevelUp with a crossover user study with 25 university-affiliated participants who had no formal AI background. Every participant creates two small projects: a spam filter and a sentiment analysis classifier, with the order of using LevelUp counterbalanced across participants so that one project is completed with LevelUp and the other without it. Analysis indicates a great improvement in project quality given usage of LevelUp based on scores using a rubric designed by the software for evaluating project quality. Conceptual understanding increases from pre-test to post-test from all students overall, even if there are no major differences between the two randomized groups. Usage data indicates no strong correlation between LevelUp accesses and final project scores. Reactions from learners indicate that most are impressed by LevelUp’s intuitiveness and motivating effect as a checklist or guide to progress, but a few are puzzled regarding some of its recommendations and ask for better examples or help [21].

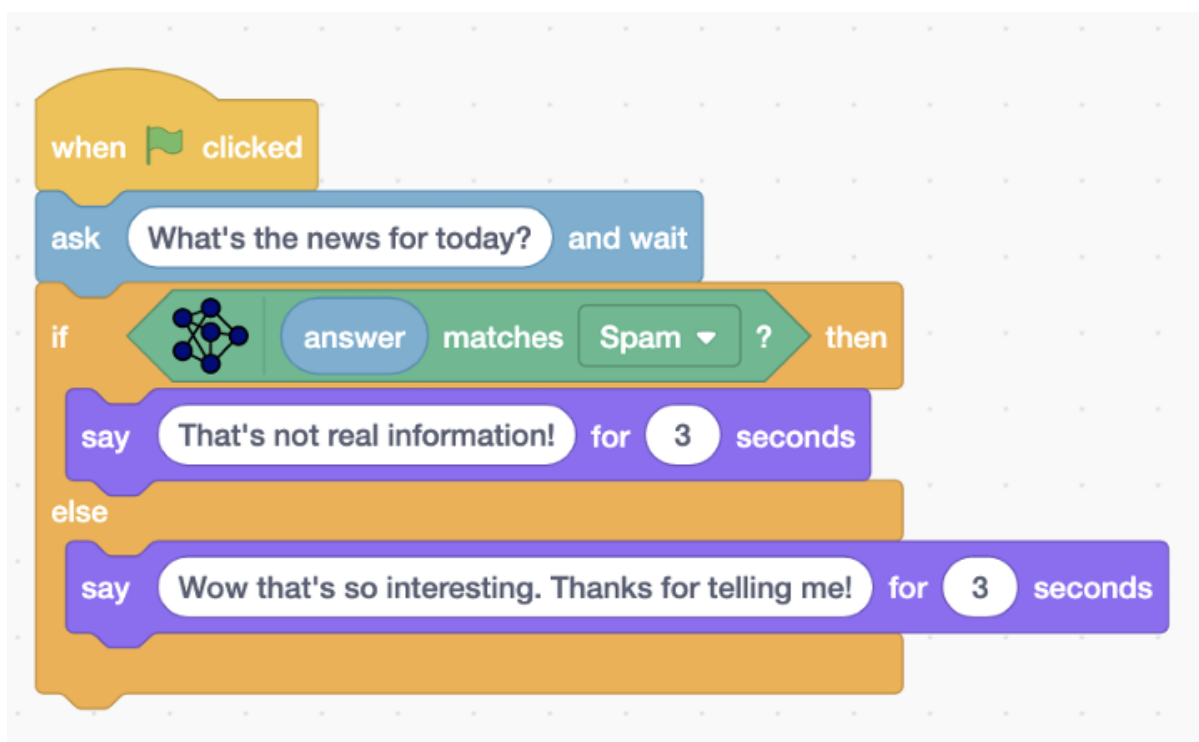


Figure 2.11: Illustrative excerpt from Reddy, Williams, and Breazeal.

Park and Shin introduce Tooe, a Scratch extension meant to help K-12 students go beyond fixed “special-purpose” AI blocks by letting Scratch talk to Python or JavaScript programs through WebSockets. The idea is simple: instead of needing a separate block

for every data source or ML feature, students use a small set of “conversational” blocks that send questions or instructions to a backend service, which can then crawl data, save files, or run machine-learning code. The extension is intentionally minimal, adding five blocks (connect, ask, tell, answer, screen), and the paper demonstrates its use through eight example projects: four big-data style apps (weather, box-office top movies, COVID dashboard, saving quiz results to CSV) and four AI apps (facial image classification, color classification, cat vs dog classification with confidence, handwriting recognition). The paper also compares Tooee to Machine Learning for Kids and Cognimates in terms of usability, extensibility, and deployability. For results, the authors report an IRB-approved teacher survey with 15 primary-school teachers who rated existing tools and Tooee before and after a short tutorial. Teachers rated Tooee higher across code complexity, usability, performance, extensibility, and deployability, with statistically significant improvements on four of the five measures. Feedback suggests teachers found the blocks intuitive and easy to start with, but some felt students would still need Scratch basics and occasional teacher guidance [17].

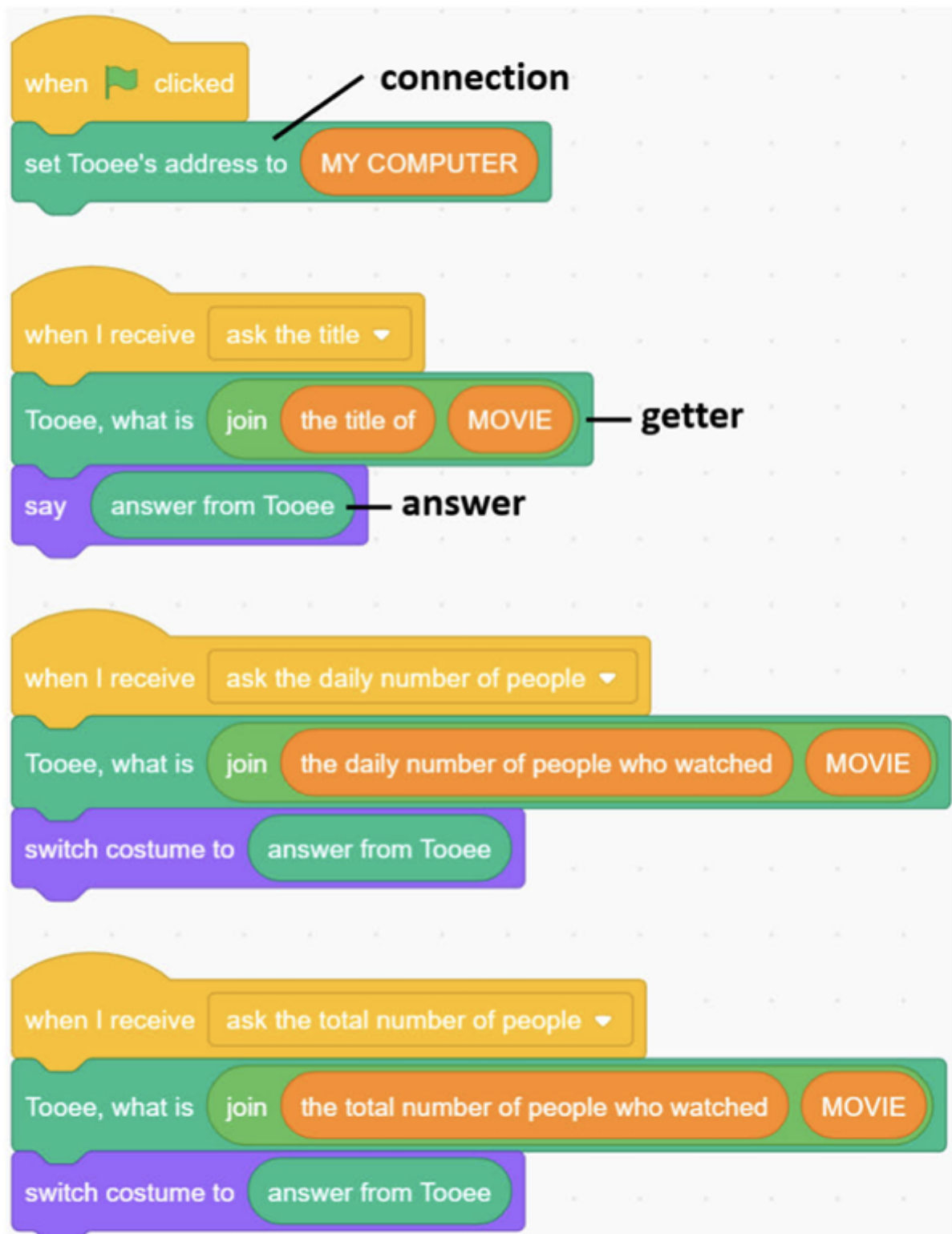


Figure 2.12: Illustrative excerpt from Park and Shin.

Andersen et al. explore what “human-centered AI” can look like in a real educational

setting where students collaborate with block-based programming. Instead of building an AI tutor and testing it, they take a research-driven approach. They run a design-based intervention over an academic year, meeting three hours a week for 16 weeks., design-based intervention where gifted lower-secondary pupils work in small groups on online STEM maker activities using micro:bit physical computing programmed in Microsoft MakeCode. Because teaching happened during COVID-19, the collaboration is captured through Zoom recordings of screen sharing, webcams, and student talk. The core contribution is an analytic lens for studying what groups produce while they work. The authors treat students' creations as "visual artifacts" made from design units such as code blocks and hardware parts, then they show how these artifacts contain different kinds of "rules." Their results identify three rule types, latent rules that are taken for granted and automated, generic rules that students apply consciously across tasks, and domain-specific rules tied to science concepts. They also find that two kinds of collaborative artifacts develop together: the evolving technical build and the evolving discussion that helps the group coordinate and learn. Finally, they propose a concrete automated scaffolding scenario, illustrated through a simulated helper agent, to shift some domain guidance from teachers to the system [3].

2.7 Deep Learning Without Text Code: Neural Networks as Blocks

Alturayef et al. present DeepScratch, a Scratch extension designed to make core deep learning ideas feel reachable for kids and high-school students by replacing text-based coding with new blocks. The paper argues that novices often struggle with deep learning because typical workflows require heavy programming plus math, so the authors extend Scratch's vocabulary with custom blocks that call TensorFlow.js behind the scenes. DeepScratch supports two pathways. The first lets learners train models using built-in datasets, specifically Iris and MNIST, and choose among Dense, RNN, and CNN architectures. Learners can change simple hyperparameters such as epochs, number of layers, and batch size, then watch training behavior through blocks that expose training accuracy, testing accuracy, and loss. The second pathway uses a pretrained object detection model, where learners turn on the camera, detect up to three objects, and read outputs like class, confidence, and object position through reporter blocks. For results, the authors report two evaluations. First, a functionality test suite with 16 test cases where all tests passed. Second, a small usability study with five students (ages 8-17). All participants completed both tasks successfully, with an average completion time of 6.18 minutes, and they reported the extension as easy to use and helpful for understanding deep learning concepts, while also requesting Arabic translations for the blocks [2].

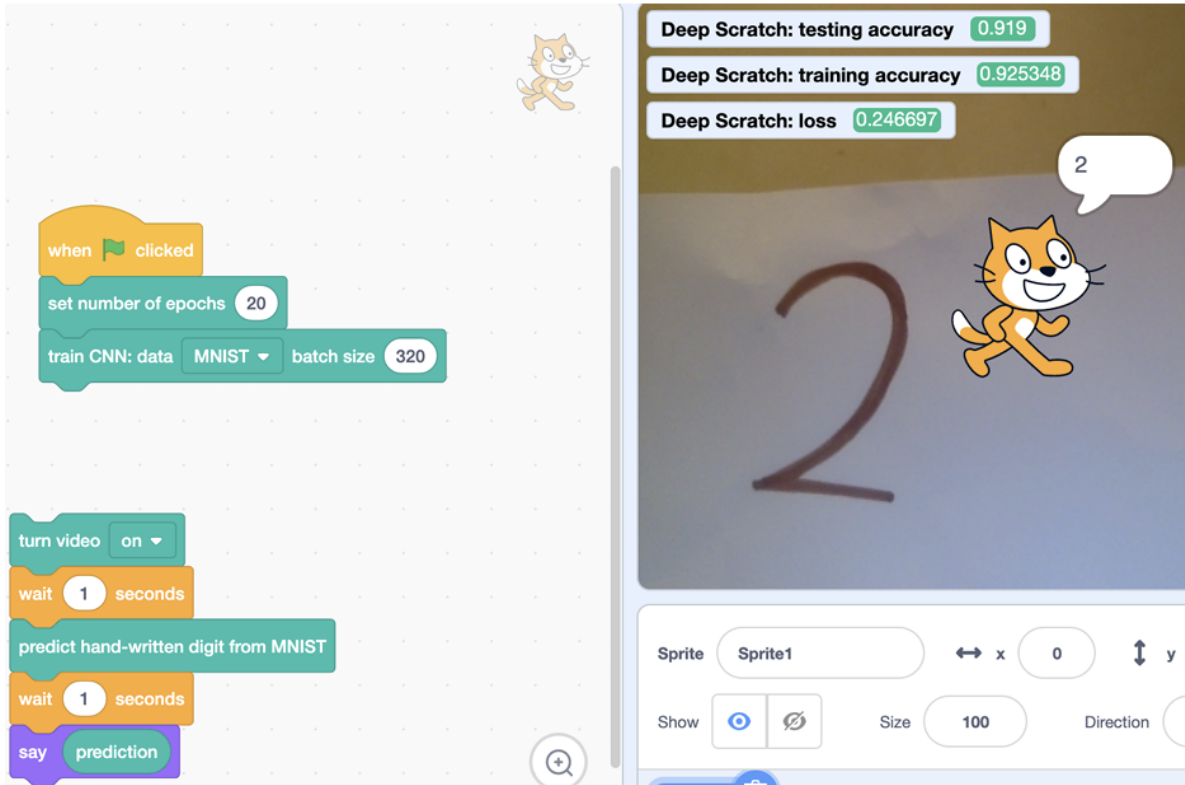


Figure 2.13: Illustrative excerpt from Alturayef et al.

2.8 Research Gaps and Motivation for VisionBlocks

From the literature review, it has been made evident that block-based and visual programming environments are rather effective ways of teaching artificial intelligence as well as machine learning to novice learners. Instead of getting confused with complex syntax or mathematical expressions, these learners are presented with the chance to learn about AI through activities such as story-building and viewing model behavior based on input data. Large-scale reviews and mapping exercises have shown that teaching students introductory knowledge about AI and machine learning in this manner leads to high engagement and motivational metrics in large numbers of educational institutions worldwide for students being introduced to AI and machine learning for the first time [4, 5, 24, 25, 23]. Block-based tools make abstract AI concepts more approachable and allow students to gain positive attitudes towards AI as they can understand the abstractions involved in AI more easily.

Despite the clear benefits of block-based AI learning environments, the literature also reveals several recurring limitations that remain unresolved. Many existing tools and learning activities emphasize using machine learning models rather than deeply understanding the processes that lead to those models, particularly the intermediate steps of the AI pipeline. Reviews and mapping studies repeatedly note that while learners can

successfully train and test models, important stages such as data preparation, feature manipulation, and preprocessing are often either hidden from learners or treated as background steps that require little reflection [4, 23]. As a result, students may develop functional skills for operating AI tools without forming a clear mental model of how early design choices influence later model behavior.

This limitation, however, becomes particularly evident in education related to computer vision, as image preprocessing is very crucial in it to get the best possible model performance. While many block-based tools allow learners to train image classifiers or recognition systems, preprocessing is often introduced in a decontextualized or optional manner, lacking an emphasis on it as an important design process. Mapping studies and tool reviews indicate that learners are typically guided toward collecting images, training a model, and observing outputs, with little structured support for experimenting with or reasoning about how operations such as resizing, normalization, or filtering affect the data before training [23, 6]. This creates the risk that preprocessing might be understood as a technical formality, not as a pivotal and inherent part of the computer vision pipeline, which directly influences the model’s behavior.

Because of these limitations, this thesis proposes VisionBlocks, a block-based learning platform tailored to promote image preprocessing as a visibly distinct and conceptually relevant part of the pipeline in computer vision. VisionBlocks focuses on having students assemble and analyze image preprocessing pipelines and make their impacts visible before beginning model training. VisionBlocks seeks to improve the students’ understanding of how preprocessing choices influence the models by structuring this process within an interactive learning workflow that encourages reasoning and analysis of image transformations. In essence, VisionBlocks directly counters a noted shortcoming in existing block-based AI tools, where models are either rapidly created and processed first before meaningfully engaging with other steps in earlier pipeline stages.

Chapter 3

Methodology

3.1 System Overview

VisionBlocks is a browser-based application and has a block-based learning approach that helps beginners conceptualize the process of computer vision in a visually interactive way. The application is targeted at first-year university students and can be used by beginners or people unfamiliar with computer vision. It does not require programming based on text. This design choice is motivated by prior work showing that block-based interfaces help beginners reason about artificial intelligence systems as a sequence of understandable operations rather than as vague black boxes, and removing textual programming from the learning process [11, 15].

From the learner’s point of view, the user interface of VisionBlocks is organized into different modules with stages within each module. Each stage is in a page that is divided into two parts. On the left side of the page, students are presented with the toolboxes that contain the blocks, and the workspace that the user will build the pipeline in through dragging and connecting the blocks that will be needed in that stage.

The right side of the interface has a set of panels that help with guidance and feedback. One such panel is the Robo companion panel, which gives students relevant context information through short dialogues that guide them during each stage, aligning with human-centered AI design approaches that emphasize embedded guidance and learner support within the interface itself [3]. There is also an output panel for students to see the results that the current pipeline has produced. The comparison view panel has an added feature in the preprocessing module with a target image side by side with the current processed image to help students see how close they are to the target pipeline.

With module 1 and the majority of the stages in module 2, adjustments made in the pipeline will result in an immediate effect on the output, allowing the students to view the effect of each process and parameter chosen. The only exception is the looping and export stage that is done on the whole dataset. This is due to the fact that adjustments made here will affect the whole dataset, not just one image, which will take time until the

process is fully complete, and the same goes for module 3, since it's about creating a model, training it, etc, which takes time to finish. Making intermediate results visible in this way reflects earlier findings that learners benefit from being able to inspect intermediate steps in machine learning pipelines [11]. Overall, the system is designed in a manner that concentrates on transparency and increments understanding by making the processing steps and their effects on the images and data clear and understandable. Through this design, VisionBlocks seeks to help the student in constructing a conceptual model in their mind of how the individual operations work together to complete a full computer vision pipeline.

VisionBlocks is implemented as a web-based system, with a Next.js and React (TypeScript) frontend that uses Blockly to enable block-based construction of computer vision pipelines. A Python backend built with FastAPI supports dataset handling, image pre-processing using OpenCV, NumPy and PIL, and model building, training, and evaluation through TensorFlow and Keras.

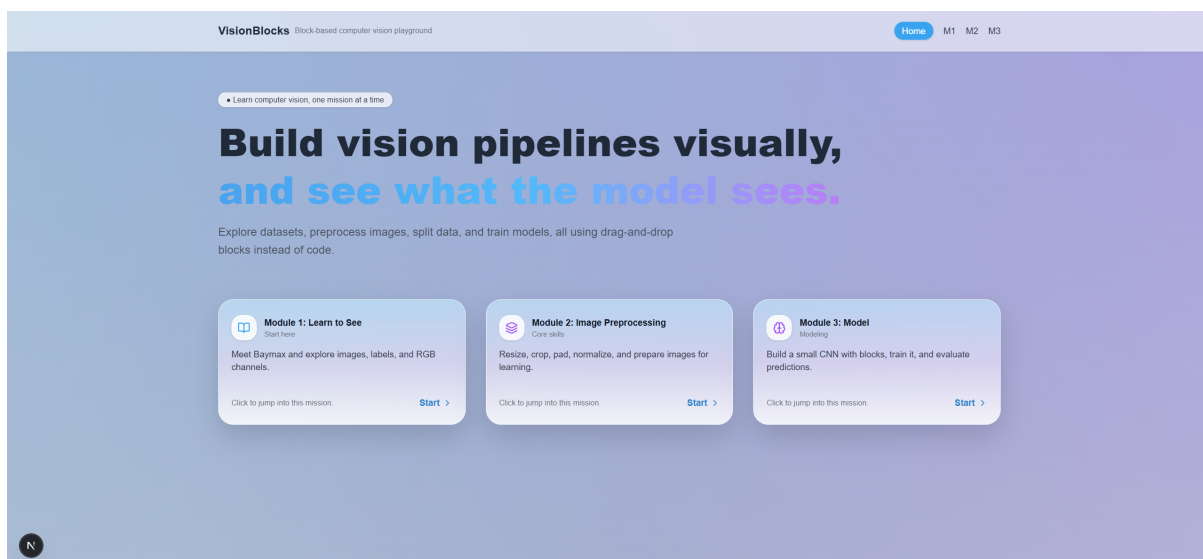


Figure 3.1: VisionBlock's main page, where the students can access the modules.

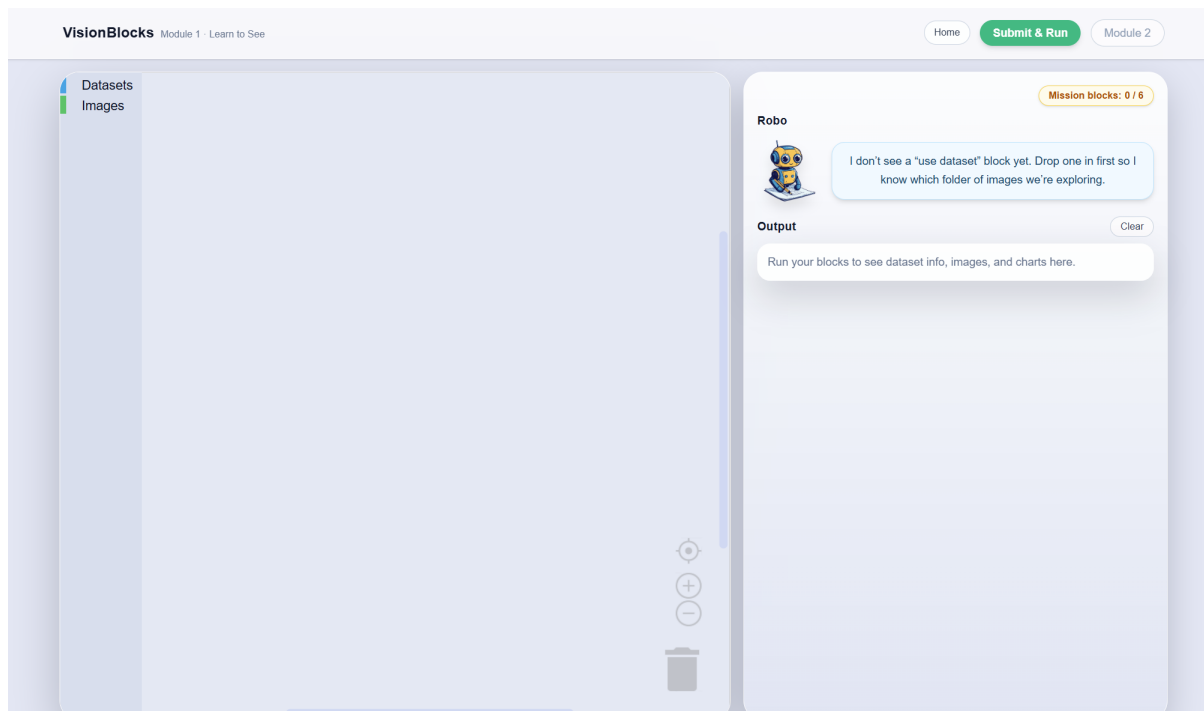


Figure 3.2: An example of a stage's page in VisionBlocks.

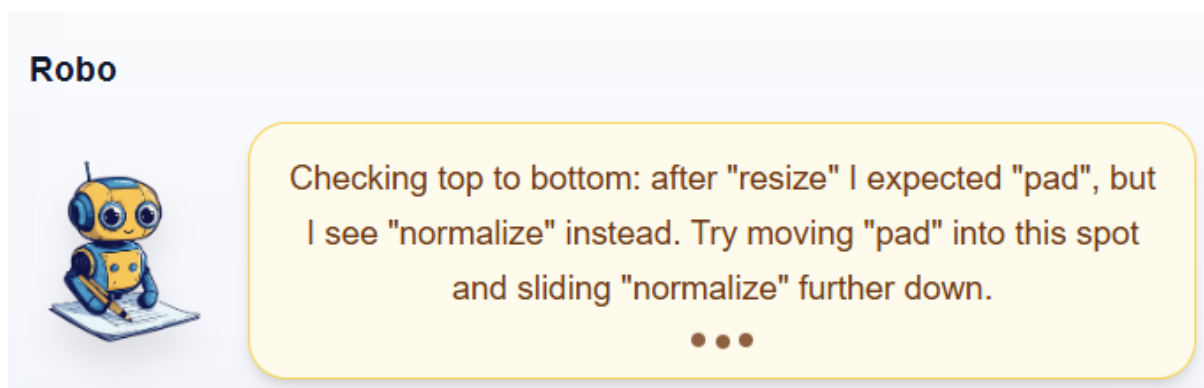


Figure 3.3: An example of Robo giving feedback when there's a misplaced block.

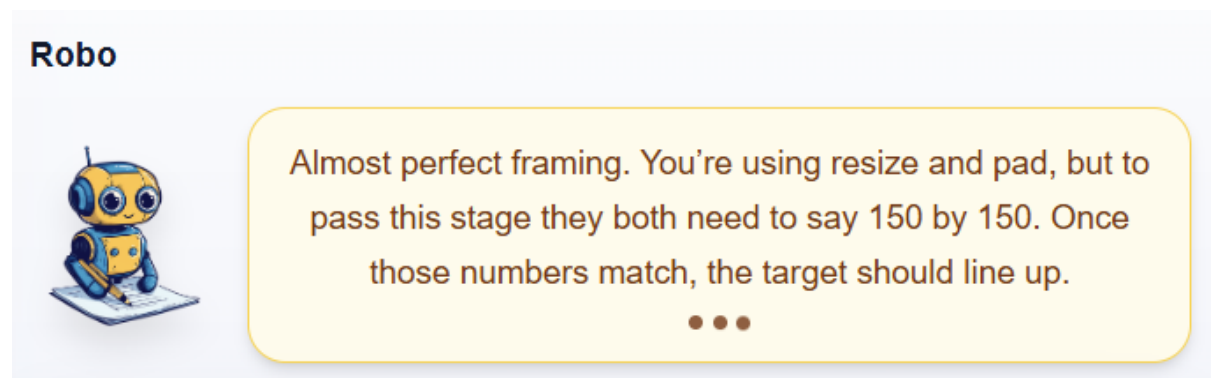


Figure 3.4: An example of Robo giving feedback when the user adds wrong parameters.

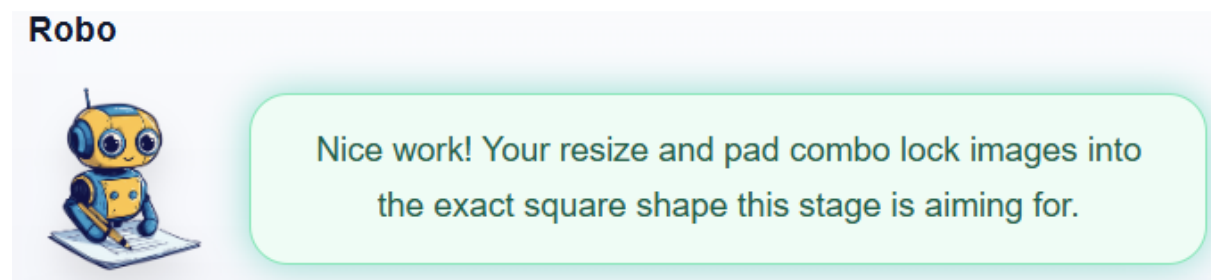


Figure 3.5: An example of Robo's dialogue When the stage is done.



Figure 3.6: An example of the output panel during module 2's stages.

3.2 System Architecture

VisionBlocks has been developed as a multi-layered web-based system intended to separate user interaction, application logic, and management of data. The overall architecture of the developed platform is shown below. The figure depicts how the proposed system has been broken down into three broad layers: a client layer, which runs inside a web browser, an application layer, which has been implemented using a backend API, and a data layer, which deals with the storage of data as well as generated artifacts. The layers have been designed to provide support for features like modularity and scalability, which, in turn, can help in keeping processes like image preprocessing, data splitting, and training tasks on the backend.

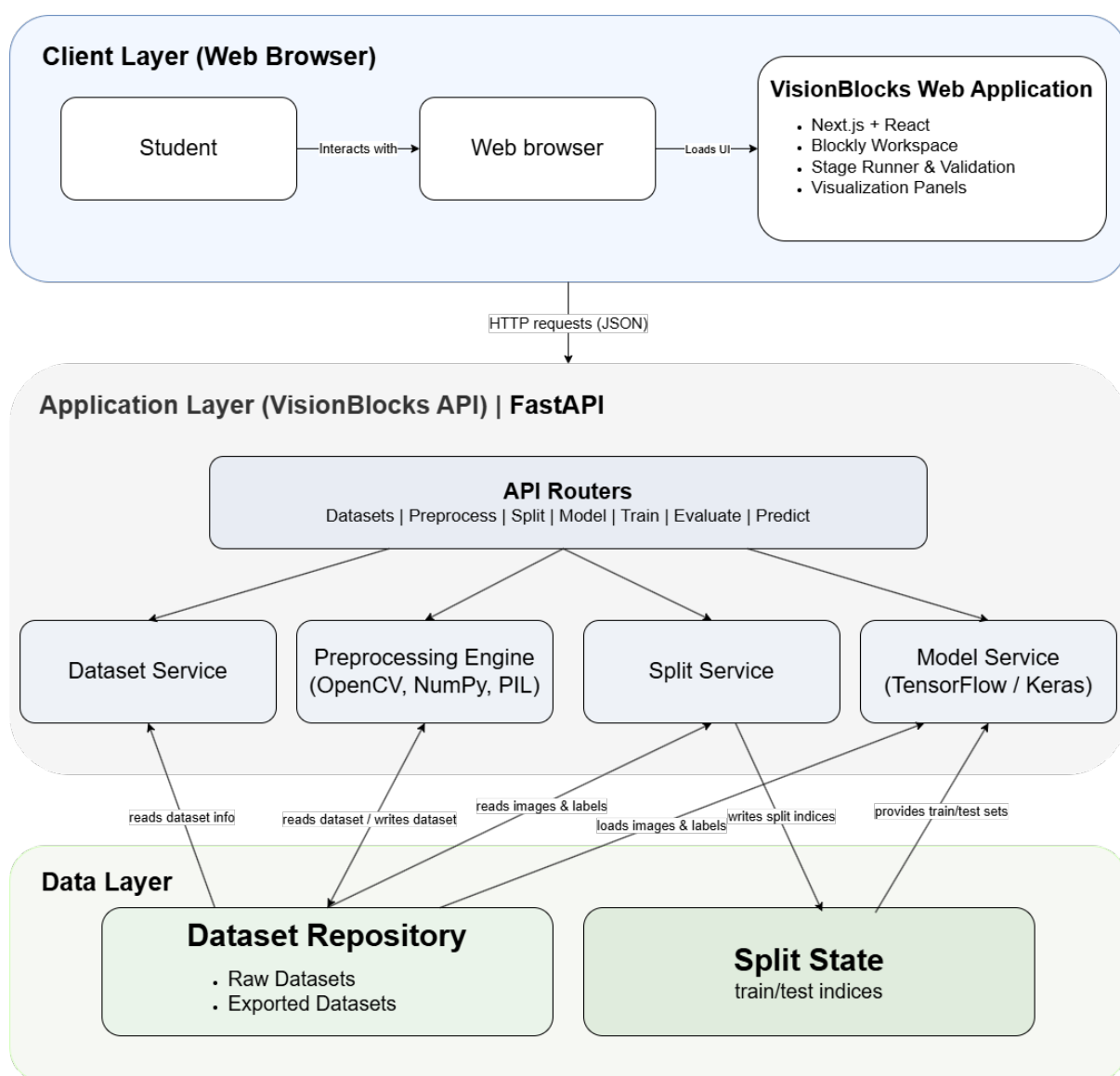


Figure 3.7: High-level system architecture of VisionBlocks.

The system follows a layered design consisting of a client layer, an application layer, and a data layer.

3.2.1 Client Layer

The client layer is the student side of the VisionBlocks application. The application runs within the web browser. This is the layer where the student's learning environment is created. Students are able to explore the datasets, create the pipelines, navigate the stages, and get the feedback while going through the stages in this layer. The application is developed using Next.js and React.

At the client layer, the students assemble computer vision pipelines through the block editor. The students do not program. They are able to drag and link blocks that denote different activities such as the choice of the dataset, the choice of the preprocessing activities, as well as the actions concerning the model like creating and training it. While designing or revising pipelines, the system is continuously checking the structure of the pipeline and provides feedback depending on it.

The client layer also plays an important role in supporting learning through feedback and guidance. Alongside the block workspace, students are provided with several panels that help them understand what they are doing and why it matters. The Robo companion panel offers short hints and explanations that react to the student's progress and mistakes, while the output panels show the results produced by the current pipeline. In the preprocessing module, this includes side-by-side visual comparisons that allow students to see how their current output differs from the target result, making the effects of each preprocessing step more concrete.

In most stages, if there are any changes within the pipeline, instant updates in the visualization appear for the students so that they can test and see the effects. In case there is a need for more computation from the system, such as dealing with an entire dataset or building a model, the client layer sends the required details to the backend and waits for a response. When the backend responds, the client layer displays the response in a visual manner that seamlessly integrates with the learning environment in a way that keeps the student away from the technical details.

3.2.2 Application Layer

The application layer is the backend of VisionBlocks that does the actual work as indicated in the pipeline that students create in the client application. The operations that are done in this application are those that involve computation or utilization of datasets. These operations include viewing the datasets, performing the necessary preprocessing operations on the images, splitting the data for the test and the train sets, as well as the training of the machine learning models. All operations are done in this layer in order for the browser application to remain responsive.

The backend is structured around services corresponding to key steps of the typical pipeline in computer vision. Each service is dedicated to handling a particular task. The dataset service enables handling dataset data and sampling images. The preprocessing engine implements image transformations described in the pipeline. The split service is responsible for managing training and testing data splits, while the model service oversees modeling, training, testing, and predictions. The system is thus simpler in terms of understanding and maintenance because it matches the logical pipeline that students are learning about.

Communication between the client layer and the application layer happens through structured requests sent over the network. When a student runs a pipeline or submits a stage, the client sends a description of the selected blocks, their order, and their parameters to the backend. The application layer interprets this description and executes the requested operations using the appropriate service. The results are then sent back to the client, where they are presented visually to the student. This separation allows students to focus on understanding the effects of their pipeline choices without being exposed to implementation details.

One of the most important aspects of the application layer is the balance it makes between responsiveness and reality. In most of the preprocessing stages, the backend performs computations on individual image samples and then sends back their results instantly, allowing for immediate visualization in the client layer. In other steps where the loads of the processes are rather high, such as when working with a full dataset or a machine learning model, the backend also performs computations in batches and then sends back their results only after completion. This is important because it introduces a student to the reality of delay in some tasks.

3.2.3 Data Layer

The data layer in VisionBlocks is in charge of the storage and organization of all data that is usable within the application. The data layer helps the application layer to support its functionality by providing it with the data of the images that are required during the learning process. The data layer helps to create structured systems where data can be reused in different modules and stages.

The dataset repository, at the center of the data layer, contains both the original datasets and any datasets generated from preprocessing by the student. Raw datasets are used in early exploration and experimentation, while the processed datasets are the output of the later stages of the preprocessing module and can be reused for data splitting and model training. By storing both forms of data, students should be able to easily tell the difference between datasets that have not been preprocessed and datasets that have been preprocessed and prepared for machine learning tasks.

Along with the storage of the datasets, the role of the data layer is to retain the split information of the dataset created in the process of dataset splitting. It stores how the

dataset will be split or divided for the training and testing, which will then be used in the model-related tasks to make sure that the correct subsets of the data are used in training and testing the models. The system remains flexible and does not copy the datasets by keeping the split information separate from the datasets themselves.

In general, the role of the data layer in VisionBlocks is supporting other layers but is crucial nonetheless. It facilitates connectivity between the different components of the framework. Additionally, this layer promotes the concepts of raw data, processed data, and derived metadata.

3.3 Module-Based Learning Structure

VisionBlocks is divided into a sequence of learning modules, each of which targets a particular area of the computer vision pipeline. By being modular in this way, VisionBlocks training provides a chance to drill and master one conceptual area before moving along to the next while keeping the tasks and conceptual ideas connected to the larger pipeline picture that needs to be fulfilled. Each module relies upon the foundations covered in previous ones and adds increasing complexity to conceptual understanding.

The current structure of the platform includes three modules. The first module can be considered an introductory module, revolving around data exploration and orientation, allowing users to get acquainted with visual data without actually transforming and modeling it. The next module deals with image preprocessing and instructs users on how to build a preprocessing step-by-step process. The third module presents an introduction to basic convolutional neural network construction and trains users to build a simple network using visual blocks. It must be noted that even if VisionBlocks can be utilized for learning every step of the computer vision process, for the experimental evaluation outlined in this article, the preprocessing module has been targeted.

Within each module, learning consists of several stages that establish specific objectives and limitations for learning. In each stage, a new idea and operation are presented for incorporation into the pipeline that has been established thus far. The methodology encourages a gradual and cumulative approach that suggests a gradual building phase for a computer vision system and not just a series of distinct points. A stage is completed when specific requirements are met, which include utilizing specific blocks, ensuring that the blocks are in the correct order, and setting the right parameters. This approach aligns with prior work on block-based learning environments that frame learning as the construction of artifacts under domain-specific constraints, where progress is guided by explicit rules and structured feedback rather than free-form exploration [3].

3.4 Module 1: Dataset Exploration

This module acts like a training mission that introduces the students to the dataset before beginning preprocessing or model training. This module is where the students will learn

how to “read” their dataset from, enabling them to get an idea about what their dataset looks like.

The students will be able to complete Module 1 by assembling a single block chain that begins with a “use dataset” block. This block allows the students to choose which dataset they want to use via a dropdown list. They are also reminded to keep all pieces of the solution in one single pipeline under this “use dataset” block, without leaving some blocks disconnected.

The necessary tasks of exploration for this particular module include three aspects of understanding the dataset. One would include an examination of the metadata of the dataset by using blocks that display the name of the dataset and the classes that the dataset has. The other aspect involves the display of the structure of the dataset by showing the counts of each class and the distribution of classes as percentages. The last aspect involves showing instances of the dataset by producing a sampling of an image of the dataset, either randomly or by index number, and breaking it down into different channels of RGB just to understand that each image is actually made up of 3 color channels, which are red, green, and blue.

An important aspect of Module 1 is that the feedback mechanism is instant, showing the output immediately after the user adds a block. Depending on the blocks that are used, the output panel might show the information cards of the dataset, distribution percentages, an image of the current sample, and a preview image of the separated red, green, or blue channels.

The module has a companion in the environment named “Robo”. Robo works by observing the student’s chain and then providing the correct hint. The hint provided has a direct relation to the mission checklist. In the mission checklist, “Robo” identifies what the student needs to complete regarding the mission, identifies where the blocks are misplaced, and recognizes where the student needs to proceed (for example, encouraging students to look at class distribution before jumping directly to sample images). For this module, since it is the first one the student will use, a guided optional tour/tutorial has also been provided, focusing on each part of the page and its usage.

For the completion of the module, a “Submit and Run” button has been provided, which upon getting pressed, will check on the blocks the student used and see if they satisfy the conditions to pass the stage. The student receives a completion notification when the chain satisfies the mission conditions. This structure keeps Module 1 lightweight, but ensures that students leave the module with a basic, concrete understanding of the dataset they will be working on in the next modules.



Figure 3.8: The blocks that get introduced in module 1.

3.5 Module 2: Image Preprocessing

Module 2 is about training students on image preprocessing as an intentional, structured, and cumulative process within the computer vision pipeline. Unlike other educational tools designed for beginners that might abstract away the preprocessing process or perform it implicitly, this module ensures that the preprocessing steps are observable and inspectable. This is achieved through the process of walking the students through various stages that cumulatively build upon the image preprocessing pipeline from raw image samples to an image dataset that is properly preprocessed. This design decision is motivated by prior educational tools that often abstract away early stages of the machine learning pipeline in order to improve accessibility, which can leave novice learners with limited insight into how models actually operate on data [16].

It consists of five essential stages, after which comes an optional bonus stage. For the assessment and experimental analysis, only stages 1 to 5 are chosen because those stages form a basic preprocessing pipeline commonly deployed in a computer vision system. All stages bring up a new concept of preprocessing, while also using the blocks that were in the previous stages. This way, the students will be able to learn about the concepts behind each block and how they should be ordered. Such ordered progression mirrors instructional designs in prior Scratch-based AI courses, where learners were guided through predefined sequences of concepts to support conceptual understanding and reinforce relationships between pipeline stages [10].

During Module 2, the students work in a similar environment to Module 1's. On the left side of the environment, they build their own connected pipeline from the preprocessing blocks, starting from their dataset and sample image. On the other side, the students get support through the Robo companion panel, and the output panel where they view the output of the blocks they used. In Module 2, there's a new panel called the target vs current image panel. The target vs current image panel shows the target image, which is the image the stage expects, and the current image, which is the image that is currently produced by the student's pipeline. This setup provides transparency to the student, where they see the impact of their preprocessing steps on the image. Providing this kind of immediate visual feedback aligns with prior Scratch-based AI learning activities, which emphasize that making intermediate computational steps visible supports learners'

understanding of the behavior of what they're working with [10]. In most stages, both the target vs current panel and the output panel have instant feedback, and change their contents immediately if the student made changes in the pipeline. Upon entering a stage, a card will pop up that contains an explanation of the concepts of the blocks that are introduced in this stage. There is a new button in the environment as well, that is a “?” button, that when pressed, will provide the explanation of the new concepts again.

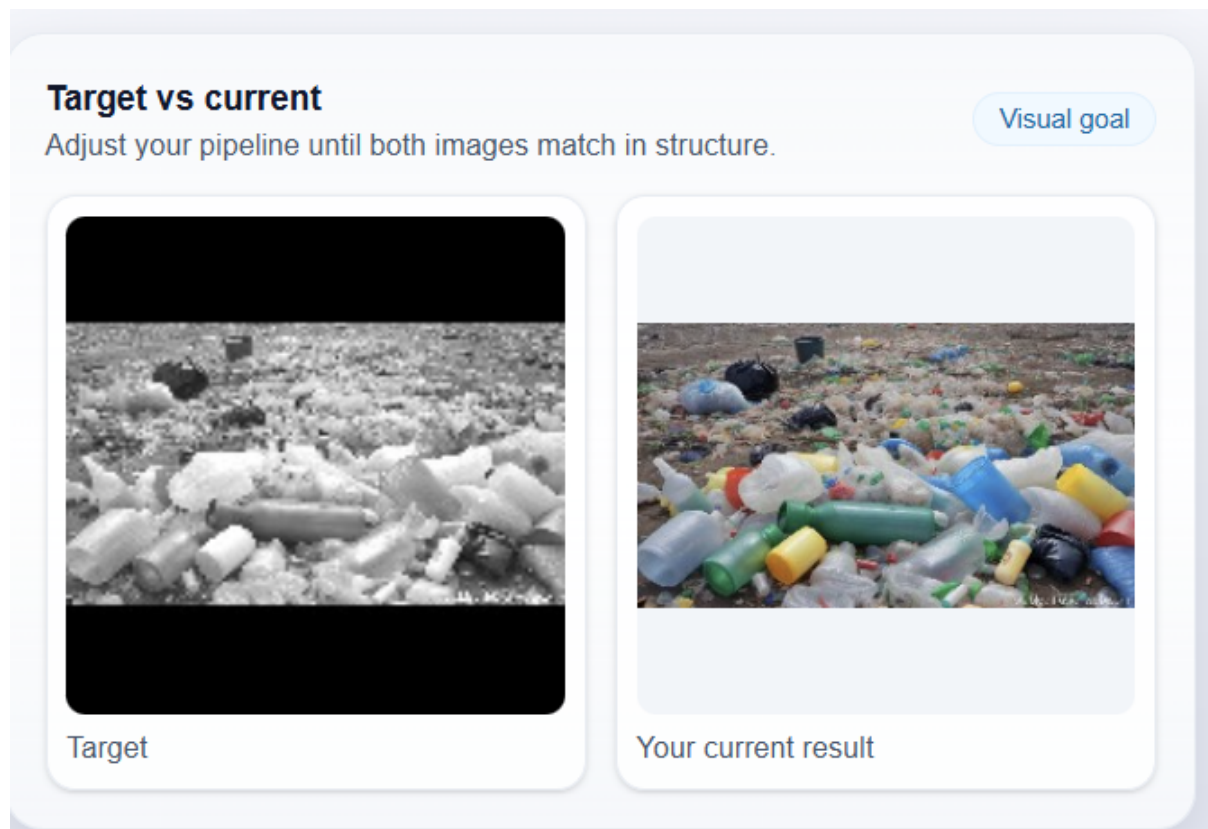


Figure 3.9: This is the new target vs current image panel.

3.5.1 Stage 1: Grayscale Conversion

The first stage introduces grayscale conversion, which serves as a necessary step in a computer vision preprocessing task. In this stage, students are tasked with converting a sample image into a grayscale format, removing color but maintaining brightness details. This step helps students understand why color information may be unnecessary or even distracting for certain computer vision tasks.

Students are required to put the grayscale block immediately after the sample image block. This satisfies the requirements of the presence as well as the positioning of this block, enforcing the concept that preprocessing steps should be done in a certain order. A target image has been provided, enabling the student to compare the generated image with the actual desired grayscale image.

3.5.2 Stage 2: Brightness, Contrast, Blur, and Sharpen

Stage 2 adds to the pipeline by including image adjustment operations like brightness, contrast, blurring, and image sharpening. Stage 2 mainly involves image cleaning and enhancement. Students are encouraged to apply tiny adjustment values, which ultimately increase the image clarity without resulting in much deformation.

At this point, students must put the blocks of brightness & contrast, and blurring & sharpening after the grayscale block. The system checks not only on the existence of the blocks, but also on their relative ordering, nudging them to think about applying low-level image operational adjustments early on. Feedback is offered by the Robo companion on a contextual basis.

In this stage, the target vs current comparison panel also plays an important part, so students can understand whether their modifications enhance readability while avoiding over-processing of the image.

3.5.3 Stage 3: Resize and Pad

Stage 3 covers the resizing and padding of the images. Students are shown that machine learning image inputs need to be of a uniform size but may be of different sizes and resolutions when received from real-world datasets. Students resize the images while maintaining the aspect ratio and then pad the resized images to a square size.

This stage requires the students to resize the images to target a resolution while preserving the aspect ratio and then pad the images to a strict 150×150 pixels. This process must be followed to the letter by the system, allowing the students to grasp the concepts of resizing, padding, and the need to accomplish the two tasks simultaneously. Visual differences between the original and target images help the students resize the images to the target resolution. The resizing and padding blocks must be placed after the blurring & sharpening block, which is the last block used in Stage 2.

This step emphasizes the importance of the fact that image processing is more than making images visually pleasant; it must also satisfy certain structural requirements.

3.5.4 Stage 4: Normalization

The emphasis of Stage 4 is on numerical data preprocessing using pixel value normalization. The concept that students are exposed to in this stage regards the fact that the original pixel values range from 0 to 255 and might cause instability in the training process if they are not normalized.

Students must add a normalization block at the end of the pipeline and set it up to scale pixel values to a 0–1 range. The program will check for the block's position and settings, and Robo will give specific error messages if an inappropriate normalization method has been chosen.

3.5.5 Stage 5: Loop and Export

The final mandatory stage transitions from experimentation involving individual images to processing at the level of datasets. At this stage, students put all the image preprocessing processes developed in previous stages inside a loop body that loops through each image in the dataset. This emphasizes that image processing pipelines need to be applied on the entire dataset, not just image samples.

Students are required to put all the preprocessing blocks within the loop body and then add an export block following the loop to create another dataset that has been processed. The system checks the internal structure of the loop to see that all required processing steps are present with the same parameters used in previous stages (considering the 150×150 image size), among others.

In contrast with previous stages, however, feedback at this stage is not wholly instantaneous, as it involves processes for multiple images rather than a single sample. This delay of feedback introduces students to the computational cost and time considerations involved when processing a full dataset, while at the same time keeping continuity with previous stages at a conceptual level.

3.5.6 Bonus Stage: Edge Detection

There is also an optional bonus stage where students are given the opportunity to try their hand at edge detection methods, perhaps using Canny or Sobel filters. This stage is not included in the experiment, as its intention is to provide an exploratory extension, not a mandatory step in preprocessing, referencing it in the stage as a special tool, not a mandatory step, and explaining that it's good to use in certain computer vision tasks, not all of them, since it can remove a lot of info if it was used on an image that contains complex backgrounds.

3.5.7 Summary of Learning Design in Module 2

Across all stages, Module 2 follows a stage-paced and constraint-driven learning methodology, where predefined structural and parameter constraints must be satisfied before progressing. This explicit decomposition of the learning process into ordered, verifiable stages reflects prior work on block-based machine learning education, which emphasizes that separating the pipeline into clear steps helps learners reason about each phase instead of treating the system as a black box [19]. The feedback is provided through a variety of mechanisms, ranging from visual comparisons to hints provided by Robo in a contextual manner. By systematically building a complete preprocessing pipeline and finally using this pipeline on a whole dataset, students form a clear mental image of how individual preprocessing operations interact within a complete computer vision workflow.

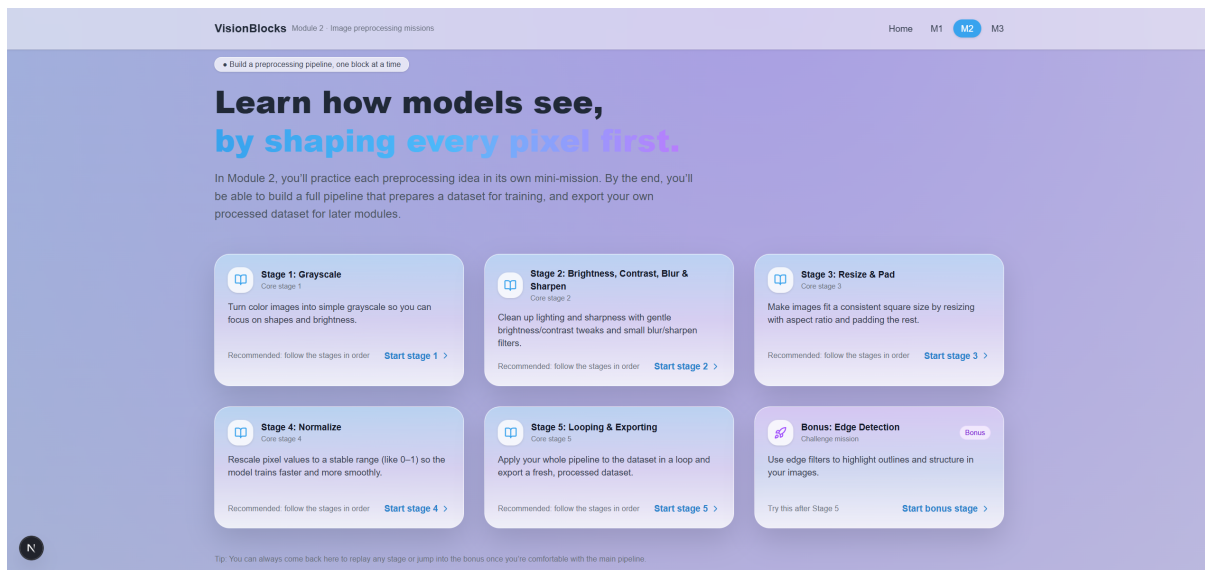


Figure 3.10: Module 2’s main page, where the students can access the different stages module 2 offers.



Figure 3.11: The blocks that get introduced in module 2.

3.6 Module 3: Model Building and Training

Module 3 teaches the students how to build a CNN model, and how to train and evaluate it. While Module 2 focuses on preprocessing the data, this module focuses on how the dataset is used to actually train and evaluate a machine learning model. Again, similar in many ways to Module 2, learning in this module is organized, cumulative, and staged so that students learn that each step builds on prior ones in meaningful ways.

The module splits into a sequence of four stages: splitting the dataset into training and testing sets, building the architecture of a CNN, training the model, and finally, the evaluation and use of the trained model for prediction. Each stage introduces a new concept of the modeling process while reusing the blocks and pipeline structure from the previous stages. In order to complete a stage, students have to use certain blocks and assemble them into a valid order within a single connected pipeline, reinforcing the notion that model development involves a strict yet meaningful order of operations.

Throughout Module 3, students work in the very same environment they have used up until now. On the left side of the interface, an elongating, connected block-based pipeline is constructed beginning with the dataset selection block and grows with each newly added component in modeling. On the right side, they get Robo companion panel guidance and see execution results in an output panel. Just as in Module 2, every stage starts with an explanation card introducing the key concepts and goals of the stage. This explanation is always available with the "?" button so that at any time students can go back to the concepts that are introduced in the current stage.

3.6.1 Stage 1: Train/Test Split

The first stage begins with the introduction of splitting a dataset into training and testing sets. Students are taught that a model will get trained on one portion of the data, then be evaluated over another portion to measure how well it generalizes on examples it hasn't seen. In this stage, the students will select a dataset and indicate the percentage of data to be used for training versus the percentage for testing, then perform the split.

To perform this step, the students will need to construct a pipeline in a certain order that starts with dataset selection, followed by configuration of the split ratio, and then the application of the split. Here, both the availability and order of these blocks are checked by the system. After execution, details in the output panel on the resulting train and test sets include sizes of the training and testing sets, showing how many images are in each set.

3.6.2 Stage 2: Creating a CNN Model

Stage 2 introduces the structure of a convolutional neural network and its image processing. Students learn how to build a CNN visually from starting a model and adding convolution and pooling layers to extract features, to finishing it off with dense layers that do classification. The goal here is to help students understand the functional roles of different layer types and the typical structuring in a CNN.

In this stage, students use the dataset split blocks from the prior stage and expand their pipeline with model-building blocks. For this stage, the pipeline needs to contain at least one convolution layer, one pooling layer, and one dense layer, placed in any valid order, succeeded by a model summary block. This model summary block gives

direct feedback, as it shows the structure of the network assembled and also indicates that the model has been created successfully, such that students can check whether their block-based design corresponds to a sensible CNN architecture.

3.6.3 Stage 3: Training of The Model

The third stage is the training of the constructed model. Students would set the number of epochs and activate the training process. In this stage, students are introduced to the concept of training as an iterative optimization procedure where the model keeps improving through more practice (more epochs) during training.

To complete this stage, students will have to add training setup and execution blocks to the existing pipeline in such a way that these blocks are placed after the model structure's blocks. During training, only the training set is used. The output panel displays training progress information such as accuracy and loss at each epoch, so that students can see how the performance of the model changes with time and relate it to their choices in configuring.

3.6.4 Stage 4: Evaluate & Predict

The final stage is model evaluation and prediction. The students evaluate the performance of the trained model by using the test set, resulting in performance metrics indicative of the model's generalization to unseen data. Besides that, the students carry out single-sample prediction by selecting any image and asking the model for its class prediction.

This stage asks students to extend the pipeline by adding an evaluation and prediction block, and also a sample image block to identify the image on which to perform the prediction. For this, the system ensures the presence and order of these blocks and Robo gives contextual hints in case any of the required elements are not in the pipeline the user built. Evaluation results, including overall accuracy and per-class accuracy, show up in the output panel together with the class predicted and confidence for individual samples. This combination lets students relate high-level performance measures to concrete prediction behavior.

3.6.5 Summary of Learning Design in Module 3

Overall, Module 3 implements a learning design that is stage-guided and constraint-driven, mirroring the real-world workflow of training and evaluating a computer vision model. Each separate stage builds upon the results of the previous one, reinforcing the dependency among data preparation, model design, training, and evaluation. All feedback is provided in the form of structural checks, contextual guidance of Robo, and detailed execution outputs. This means that by the end of the module, students have built and run an end-to-end model pipeline that provides an integrated overview of how preprocessed image data is turned into actual, trained models and actionable predictions.

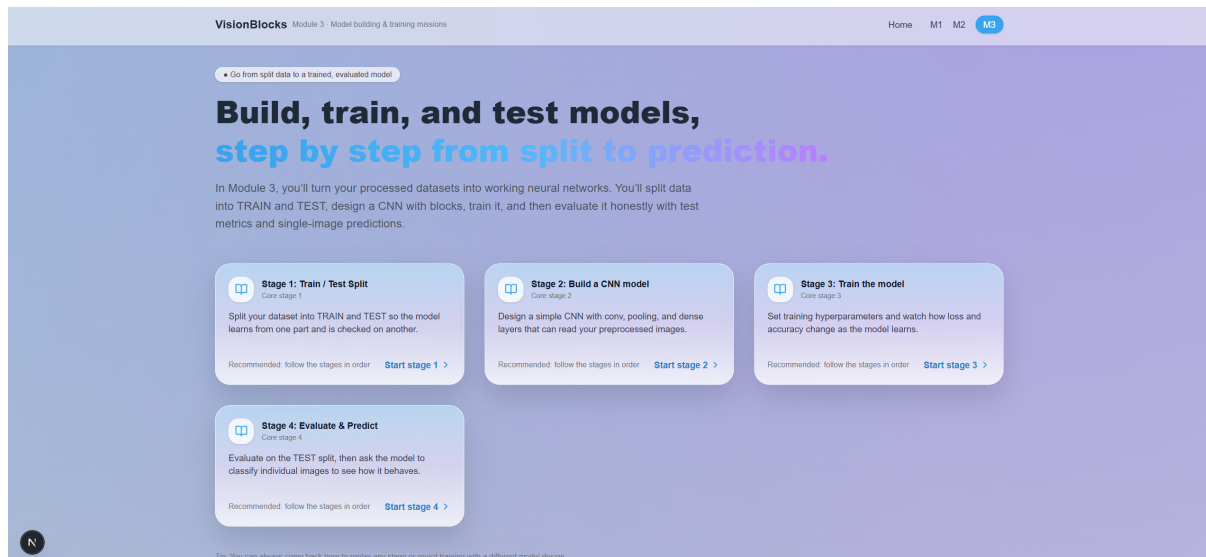


Figure 3.12: Module 3's main page, where the students can access the different stages module 3 offers.

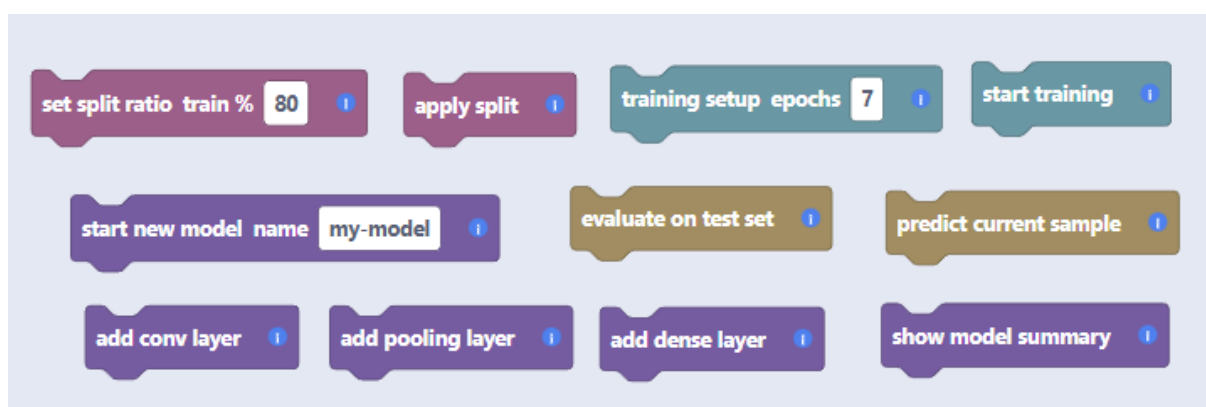


Figure 3.13: The blocks that get introduced in module 3.

Chapter 4

Experimental Design

4.1 Participants

The experiment's participants consisted of 30 first-year university students, divided equally into two groups. These participants were drawn from various fields of study, including computer science and engineering, and were at an early stage in their undergraduate career. None of the subjects had any experience in computer vision. This ensured that the study was on novice participants, who form the target group that VisionBlocks intends to cater to.

The participants were equally allocated into two groups: the experimental group and the control group, comprising 15 students in each group.

4.2 Experimental Conditions

The experimental design used in this research was between-groups, focusing on acquiring concepts related to image preprocessing.

Experimental Group (VisionBlocks Group): These students used VisionBlocks Module 2 (Image Preprocessing). They worked through Stage 1 to Stage 5 of the module, including concepts like grayscale, brightness, contrast, blurring, sharpening, resizing, padding, normalizing, and finally looping to export the preprocessed dataset.

Control Group (Video-Based Instruction Group): The students who belonged to the control group were shown a video explanation that included the same preprocessing ideas and steps that had been introduced in Module 2's stages from stage 1 to 5. The video resembled the conceptual progression of the stages of VisionBlocks but did not include actual interaction or construction using the blocks.

In both cases, both groups were presented with the material in one sitting, and the experiment was conducted in a normal room.

4.3 Experimental Procedure

The process included the same overall framework for both groups:

Pre-test: Before the participants were exposed to the learning content, a pre-test was carried out to check the understanding level of the participants regarding the concepts related to image preprocessing techniques.

Learning Phase: The experimental group progressed through the stages of VisionBlocks Module 2 in an interactive manner, while the control group was viewed the instructional video, which dealt with the same issues.

Post-test: After participation in the learning activity, a post-test, containing the same questions in the pre-test, was administered.

This facilitated a direct comparison of conceptual understanding levels at the beginning and end of the learning intervention because similar questions were used in both assessments.

Engagement Questionnaire: Participants' engagement was determined using a short questionnaire. The questionnaire was administered after the learning activity. The questionnaire gathered information on the perceived engagement and interest in the subject. This questionnaire got administered to all participants in order to evaluate the difference in engagement in both groups.

System Usability Scale (SUS): Apart from learning outcomes, the usability of the system for the experimental group is measured using the System Usability Scale or SUS. Only participants in the experiment that have used VisionBlocks completed the survey questionnaire, since the survey is meant for assessing an interactive system. The questionnaire was administered after the learning activity. The control group did not complete the survey since they have not had an interaction with VisionBlocks.

4.4 Focus Group Feedback and System Refinement

Prior to the actual experiment, a small focus group was conducted to collect some initial qualitative feedback about VisionBlocks to identify possible usability or clarity problems that might impact learning. The focus group included four first-year university students, which are the same age of the audience VisionBlocks targets. None of the participants in the focus group took part in the actual experimental study.

The focus group was conducted in an informal setting. Participants were introduced to the system and were asked to explore VisionBlocks freely, with a particular focus on Module 2 (Image Preprocessing). They were encouraged to test the system and provide any feedback they had regarding clarity, guidance, and usability. Feedback was collected through open discussion and observation during system use, rather than through a structured questionnaire.

Several recurring points were raised by the participants. First, they indicated that the target versus current image comparison panel was not immediately clear when entering the first preprocessing stage. To address this, a short tutorial was added that appears when users enter the first stage of Module 2, explaining the purpose of the target and current image panels.

Second, participants said that when new preprocessing blocks were added, it wasn't always clear how they fit in with the ones that had already been added. In response, clear instructions were added when new blocks were introduced to make it clear that they should be used after the blocks from earlier stages.

Third, participants said that the response they got after finishing a stage could be better. The system was changed to show a clear "Next Stage" button instead of a confirmation message like "nice, got it." This made it easier to move through the module.

Fourth, participants expressed that explanations of the concepts introduced in each stage should be presented immediately upon entering the stage. As a result, conceptual explanations were adjusted to appear automatically at the beginning of each stage.

Finally, participants recommended explicitly informing users that blocks introduced in earlier stages should continue to be used, and in the same order, to reflect the correct sequence of preprocessing steps within the computer vision pipeline. This instruction was added to reinforce the intended structure of the pipeline and to guide students toward correct block assembly.

Responses from the focus group were integrated into VisionBlocks before the beginning of the experimental study to improve the clarity and usability of the system without changing its original goal before evaluation.

4.5 Measured Outcomes

The experiment was aimed at the following outcomes:

- Learning gains, determined through scoring before and after testing for various group comparison settings.
- Engagement, measured using a post-activity questionnaire administered to participants.
- System usability, measured using the System Usability Scale questionnaire for the VisionBlocks group.

The data was collected using Google Forms questionnaires.

The experiment's aim: The aim of the experiment is to test the null hypothesis. The null hypothesis states that there is no difference between using VisionBlocks and using the traditional methods, like watching a video, in terms of the learning experience. Two null hypotheses were tested: the first null hypothesis (H1) states that there is no difference in the learning gain between the experimental group and the control group. The second null hypothesis (H2) states that there is no difference in the engagement level of the participants in the experimental group and the control group.

Chapter 5

Test and Results

A total of 30 first-year students at a university took part in this experiment. All participants had little to no experience at computer vision. Those participating were equally divided into two groups, an experimental group consisting of 15 participants, alongside a control group also consisting of 15 participants. These two groups were presented with two different ways of learning, focusing on concepts about image preprocessing in computer vision.

Participants in the experimental group learned by using VisionBlocks. The participants went through Stages 1 to 5 in Module 2, in which the following topics related to preprocessing a dataset are introduced: grayscale conversion, brightness and contrast adjustments, blurring and sharpening, resizing and padding, normalization, and looping to export the fully preprocessed dataset. The Control group participants learned the same topics but through a video instead.

Both learning activities were conducted in a normal room and completed in a single session. The duration of the learning phase was approximately the same for both groups. To compare how effective the two learning methods were, the learning gains and engagement levels were measured for both groups. The independent sample t-test was applied using R (a programming language used for statistical analysis) to determine whether differences between the two groups were significant or not. VisionBlocks' usability as a system as well was measured using the System Usability Scale (SUS). Google forms was used to create the questionnaires and collect the data.

5.1 Learning Gain

Learning gain refers to how much the participants improved their understanding as a result of the learning activity. In this study, learning gain was measured using a pre test and post test design, where the pre test measures how much they know about the topic before the activity, and the post test measures how much they learned about the topic after the activity.

Procedure: Before the learning activity, all participants completed a pre test designed to assess their prior knowledge of image preprocessing concepts. The tests covered key preprocessing concepts addressed in the learning phase. The test consisted of 9 MCQ questions that covered the concepts in module 2's stages 1 through 5. After completing the assigned learning activity, participants completed a post test that contains the same questions in the pre test. Learning gain was calculated by subtracting each participant's pre test score from their post test score. This allowed for a direct comparison of knowledge improvement across both groups.

Results: Learning gain scores for the experimental and control groups were compared using an independent sample t-test. The results indicated that there was no statistically significant difference in the learning gain of the two groups. The p-value obtained was $p = 0.09018$, which was greater than the significance level of 0.05. This means that both learning strategies resulted in comparable learning gains in terms of the understanding of image preprocessing concepts. Thus, we cannot reject the first hypothesis (H1), which claimed that there is no difference in the learning gain between the two groups.

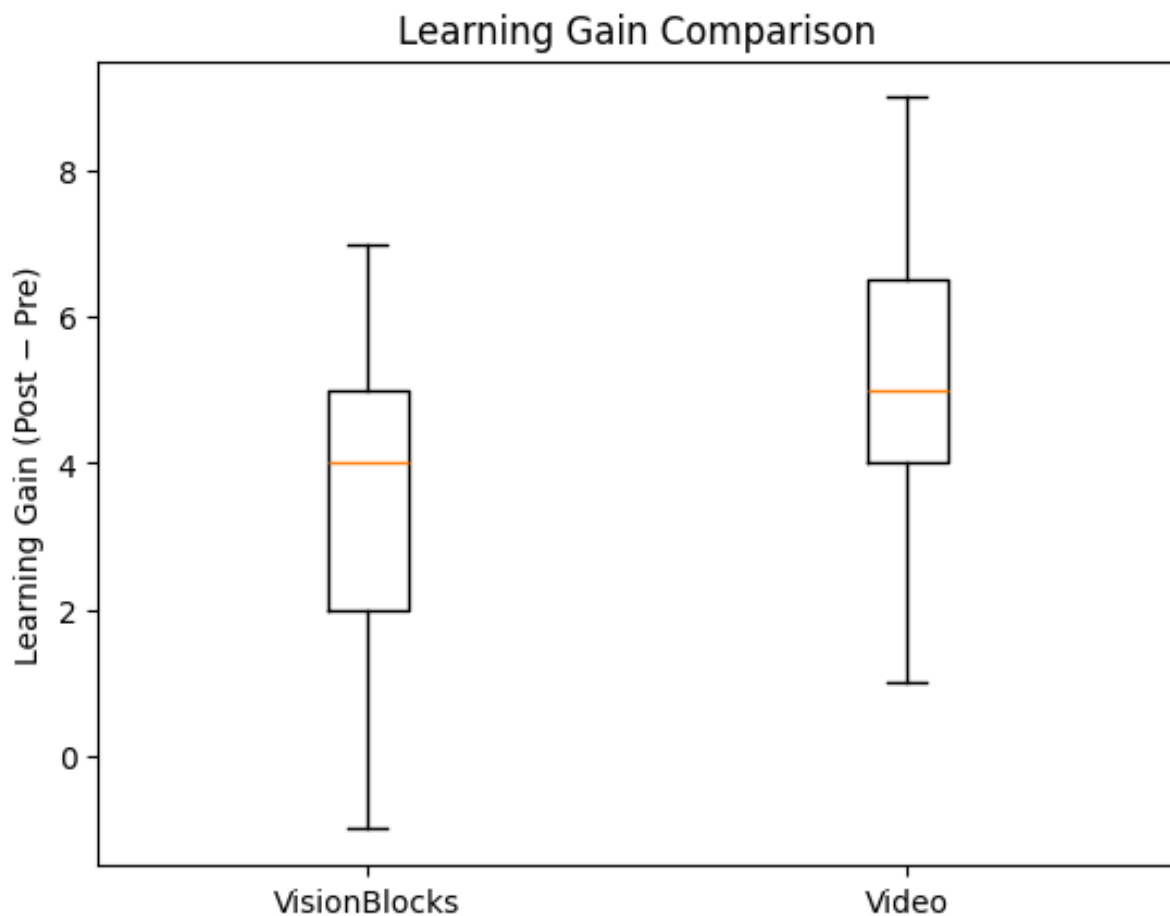


Figure 5.1: A graph showing a comparison between both groups' learning gains

5.2 Engagement

Engagement refers to the level of involvement, interest, and attention experienced by participants during the learning activity. In the context of interactive and educational systems, engagement is often considered an important factor influencing motivation and learning experience.

Procedure: Engagement was measured through a questionnaire that gets filled out after the learning activity. The format of the questionnaire included nine items on a five-point Likert scale ranging from strongly disagree to strongly agree. The items addressed aspects such as perceived control, enjoyment, interest, curiosity, clarity of progression, effort, frustration, and boredom.

For each participant, an overall engagement score was determined by taking the average of all nine questionnaire items. This questionnaire was administered to the experimental and control groups in order to be able to compare the level of engagement in the activity between the two groups.

Results: The results indicate that the experimental group showed higher levels of engagement compared to the control group. The experimental group had an average of 4.07 in the level of engagement, contrary to the control group that attained an average of 3.32, with a difference between the two group of 0.75. This difference and the independent sample t-test show a significant improvement in the level of engagement for the participants who used VisionBlocks ($p < 0.05$). Thus, the second hypothesis (H2) stating that there is no difference in the engagement between the experimental group and the control group is rejected.

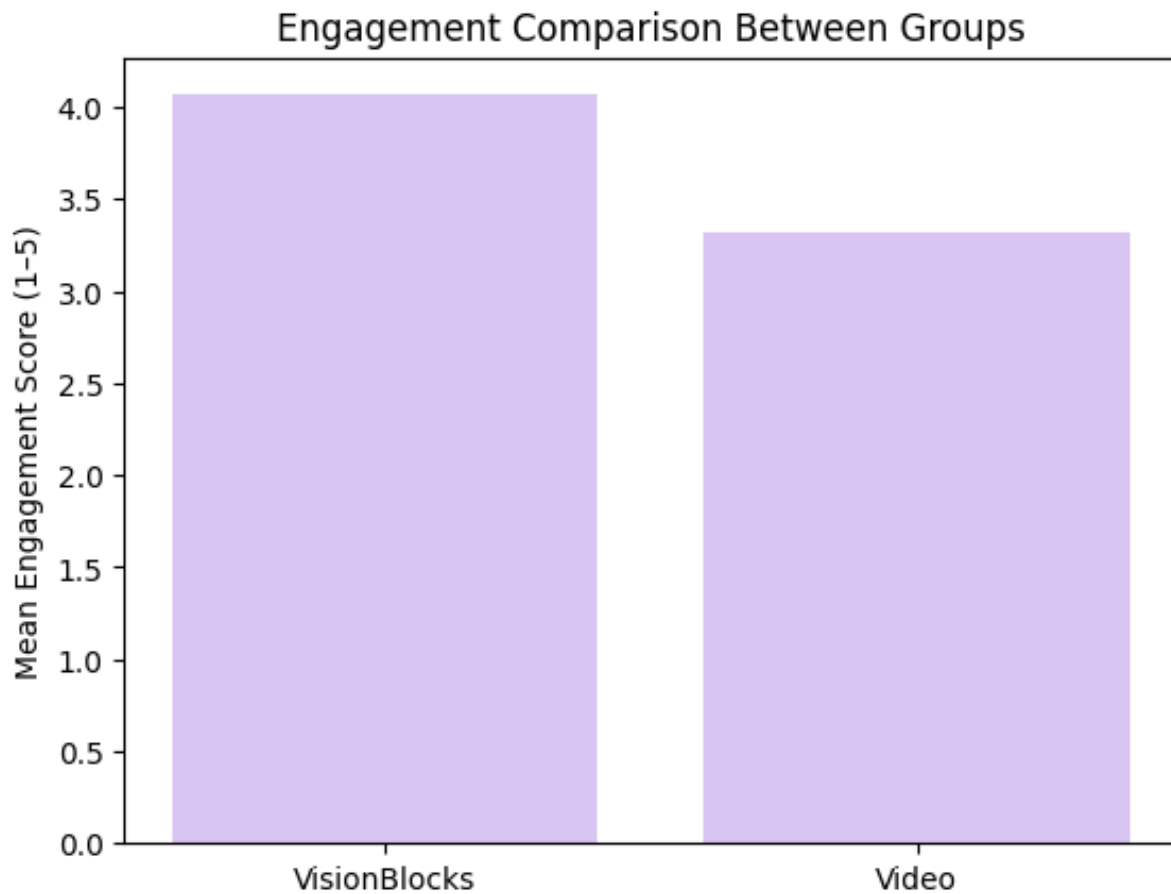


Figure 5.2: A graph showing a comparison of the engagement between both groups

5.3 System Usability

System usability reflects how easy and comfortable a system is to use from the user's perspective. To evaluate the perceived usability of VisionBlocks, the System Usability Scale (SUS) was used.

Procedure: Usability was measured using the System Usability Scale, which consists of ten items rated on a five point Likert scale ranging from strongly disagree to strongly agree, with the odd items (item 1, 3, etc.) as positive statements and even items (item 2, 4, etc.) as negative statements. Responses were scored using the standard SUS procedure, where item scores are converted and summed, then multiplied by 2.5 to produce a final score ranging from 0 to 100.

Only participants in the experimental group completed the SUS questionnaire, since the control group did not use VisionBlocks.

Results: The experimental group scored an average of 74 on the SUS, which is above the commonly cited average score of 68.

The SUS results are consistent with the engagement findings, suggesting that participants generally found the system easy to use.

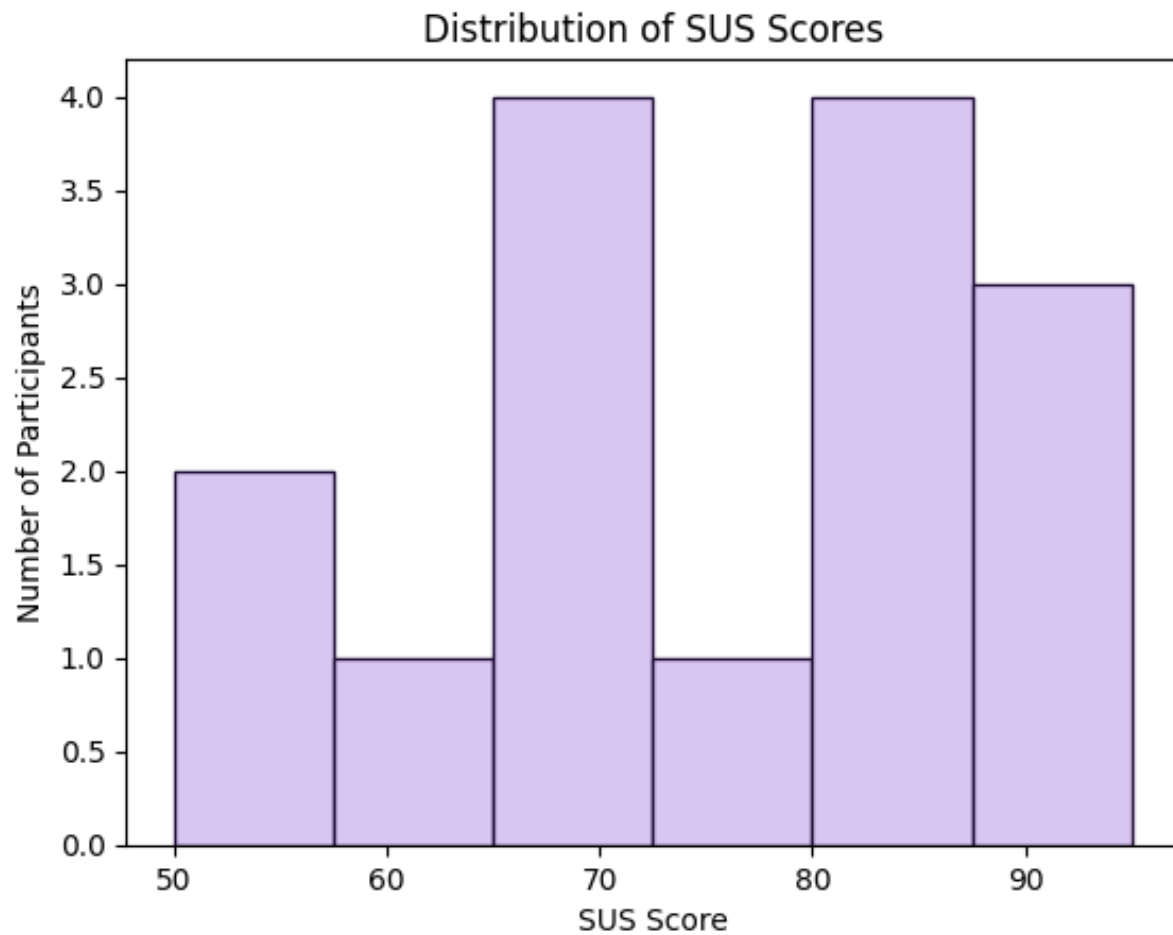


Figure 5.3: A graph showing the distribution of SUS scores

Chapter 6

Conclusion and Future Work

The goal of this thesis was to examine how computer vision can be made more accessible to novice learners by ensuring the preprocessing phase is made more significant and understandable. The preprocessing phase is an important part of computer vision. However, it is usually treated as a technical step that receives less emphasis than the modeling phases, which can lead learners to apply preprocessing steps without fully understanding their purpose or impact. VisionBlocks was developed as a solution to counter this problem by focusing on the preprocessing steps and making it interactive instead of being a preliminary step that learners rush through before reaching the modeling phase.

VisionBlocks is a block-based learning environment targeting first-year university students with little or no prior experience in computer vision. The learning progresses in a series of stages targeting the construction of a complete computer vision pipeline, from exploratory analysis to creating CNN models, so the students can observe the impact of each step and understand how the operations interact with each other. The learning system encourages students to pay more attention to the preprocessing step by emphasizing visibility and providing immediate feedback.

The experimental design used a between-group design to compare VisionBlocks with a video-based learning approach. Results showed no statistically significant difference between the two groups in terms of learning gain, indicating that both learning approaches were comparably effective for teaching preprocessing concepts. However, students who used VisionBlocks reported significantly higher engagement, and the system usability test's results indicated that the system was generally easy to use.

Despite these findings, there are some limitations. It is not possible to draw conclusions about long-term learning or how much the participants remember after some time passes because the experiment only had 15 participants in each group and one session. Also, the evaluation only looked at the preprocessing steps and not how this knowledge affects later stages of the computer vision pipeline.

For future work, it would be valuable for researchers to conduct longer-term studies with larger and more diverse groups of students and provide post-tests later in order to

understand how well they remember what they learned. Similarly, it would be useful to determine whether the students' reasoning about training and testing models improves as students learn more about preprocessing. From a design perspective, future versions of VisionBlocks could consider more flexible forms of guidance that afford students more freedom as they gain more confidence in their knowledge. They may also include richer feedback so that the system can be more beneficial to the students' understanding.

In general, this work shows that treating image preprocessing as a top priority for learning is both useful and possible. VisionBlocks shows that it is possible to make a part of the computer vision pipeline that is often under-emphasized more visible and interesting without hurting learning outcomes. This is a good direction for future educational tools in this area.

Appendix

List of Figures

2.1	Illustrative excerpt from Estevez et al.	7
2.2	Illustrative excerpt from Jatzlau et al.	8
2.3	Illustrative excerpt from Quiroz and Gutierrez.	9
2.4	Illustrative excerpt from Kahn et al.	11
2.5	Illustrative excerpt from Kahn and Winters.	12
2.6	Illustrative excerpt from Jordan et al.	14
2.7	Illustrative excerpt from Carney et al.	15
2.8	Illustrative excerpt from Pacheco et al.	16
2.9	Illustrative excerpt from Rao et al.	18
2.10	Illustrative excerpt from Du et al.	19
2.11	Illustrative excerpt from Reddy, Williams, and Breazeal.	20
2.12	Illustrative excerpt from Park and Shin.	22
2.13	Illustrative excerpt from Alturayeif et al.	24
3.1	VisionBlock’s main page, where the students can access the modules. . .	28
3.2	An example of a stage’s page in VisionBlocks.	29
3.3	An example of Robo giving feedback when there’s a misplaced block. . .	29
3.4	An example of Robo giving feedback when the user adds wrong parameters. .	30
3.5	An example of Robo’s dialogue When the stage is done.	30
3.6	An example of the output panel during module 2’s stages.	30
3.7	High-level system architecture of VisionBlocks.	31
3.8	The blocks that get introduced in module 1.	36
3.9	This is the new target vs current image panel.	37

<i>LIST OF FIGURES</i>	59
3.10 Module 2's main page, where the students can access the different stages module 2 offers.	40
3.11 The blocks that get introduced in module 2.	40
3.12 Module 3's main page, where the students can access the different stages module 3 offers.	43
3.13 The blocks that get introduced in module 3.	43
5.1 A graph showing a comparison between both groups' learning gains . . .	50
5.2 A graph showing a comparison of the engagement between both groups .	52
5.3 A graph showing the distribution of SUS scores	53

Bibliography

- [1] A. Agassi, H. Erel, I. Y. Wald, and O. Zuckerman. Scratch nodes ml: A playful system for children to create gesture recognition classifiers. In *CHI Extended Abstracts (CHI EA '19)*, pages 1–6, 2019.
- [2] N. Alturayef, N. Alturaief, and Z. Alhathloul. Deepscratch: Scratch programming language extension for deep learning education. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 11(7):642–650, 2020.
- [3] R. Andersen, A. I. Mørch, and K. T. Litherland. Collaborative learning with block-based programming: Investigating human-centered artificial intelligence in education. *Behaviour & Information Technology*, 41(9):1830–1847, 2022.
- [4] R. Ayman and N. Sharaf. Bringing ai to life: A review of ai block-based learning tools. Manuscript.
- [5] B. Batni, S. N. Junaini, J. Sidi, W. A. Mustafa, and Z. I. A. Ismail. Current research trends of scratch block-based programming for k-12: A systematic review. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 51(2):138–152, 2025.
- [6] M. Carney, B. Webster, I. Alvarado, K. Phillips, N. Howell, J. Griffith, J. Joneejan, A. Pitaru, and A. Chen. Teachable machine: Approachable web-based tool for exploring machine learning classification. In *CHI Extended Abstracts (CHI EA '20)*, pages 1–8, 2020.
- [7] S. Druga. Growing up with ai: Cognimates—from coding to teaching machines. Master’s thesis, Massachusetts Institute of Technology, Cambridge, MA, 2018.
- [8] R. Du et al. Rapsai: Accelerating machine learning prototyping of multimedia applications through visual programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '23)*, pages 1–23, 2023.
- [9] J. Estevez, G. Garate, and M. Graña. Gentle introduction to artificial intelligence for high-school students using scratch. *IEEE Access*, 7:179027–179036, 2019.
- [10] J. Estevez, G. Garate, J. M. Lopez-Guede, and M. Graña. Using scratch to teach undergraduate students’ skills on artificial intelligence. arXiv:1904.00296, Mar 2019.

- [11] S. Jatzlau, T. Michaeli, S. Seegerer, and R. Romeike. It's not magic after all: Machine learning in snap! using reinforcement learning. In *2019 IEEE Blocks and Beyond Workshop (BB)*, pages 37–41, 2019.
- [12] B. Jordan, N. Devasia, J. Hong, R. Williams, and C. Breazeal. Poseblocks: A toolkit for creating (and dancing) with ai. In *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21)*, 2021.
- [13] K. Kahn, R. Megasari, E. Piantari, and E. Junaeti. Ai programming by children using snap! block programming in a developing country. In *EC-TEL Practitioner Proceedings*, 2018.
- [14] K. Kahn, N. Naveen, R. Prasad, and G. Veera. Ai snap! blocks for speech input and output, computer vision, word embeddings, and neural net creation, training, and use. In *Proceedings of Constructionism 2021*, pages 1–10, 2021.
- [15] K. Kahn and N. Winters. Child-friendly programming interfaces to ai cloud services. Unpublished/technical report.
- [16] J. Pacheco, J. Ferreira, H. Tavares, and M. Miranda. Machine learning tool for kids: A contribution to teaching computational thinking in schools, Sep 2023. ResearchGate Preprint.
- [17] Y. Park and Y. Shin. Tooe: A novel scratch extension for k-12 big data and artificial intelligence education using text-based visual blocks. *IEEE Access*, 9:149630–149646, 2021.
- [18] Y. Park and Y. Shin. A block-based interactive programming environment for large-scale machine learning education. *Applied Sciences*, 12:13008, 2022.
- [19] P. Quiroz and F. J. Gutierrez. Scratch-nb: A scratch extension for introducing k-12 learners to supervised machine learning. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education (SIGCSE '24)*, pages 1–7, 2024.
- [20] A. Rao, A. Bihani, and M. Nair. Milo: A visual programming environment for data science education. In *2018 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2018.
- [21] T. Reddy, R. Williams, and C. Breazeal. Levelup: Automatic assessment of block-based machine learning projects for ai education. In *2022 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 133–142, 2022.
- [22] J. D. Rodríguez-García, J. Moreno-León, M. Román-González, and G. Robles. Learningml: A tool to foster computational thinking skills through practical artificial intelligence projects. *RED. Revista de Educación a Distancia*, 20(63):Art. 7, 2020.
- [23] C. G. von. Wangenheim, C. R. Hauck, J. F. S. Pacheco, and M. F. B. Bueno. Visual tools for teaching machine learning in k-12: A ten-year systematic mapping. *Education and Information Technologies*, 30:1–35, 2025.

- [24] I. H. Y. Yim and J. Su. Artificial intelligence (ai) learning tools in k-12 education: A scoping review. *Journal of Computers in Education*, 12(1):93–131, 2025.
- [25] I. H. Y. Yim and J. Su. Artificial intelligence literacy education in primary schools: A review. *International Journal of Technology and Design Education*, 2025.
- [26] C. Zhou. Aibee: Ai-driven block-coding educational environment for teaching, learning, and prototyping ai. In *IEEE Global Engineering Education Conference (EDUCON)*, pages 1–8, 2023.